CS512
Machine Learning

Course Logistics and
Introduction to Machine Learning

Sabanci University
Öznur Taştan

Today's Schedule

- Course logistics and outline
- An overview of machine learning

Today's Schedule

- Course logistics and outline
- An overview of machine learning

Course Logistics

- **Instructor**
  - Öznur Taştan
    - otastan@sabanciuniv.edu
    - Office hours: Thu 14:40 – 15:30, FENS 2058
- **Lecture hours:**
  - Wed 14:40 – 15:30
  - Thu 10:30 – 12:30

Short Bio

B.Sc.



M.Sc. & Ph.D.

Carnegie Mellon University
School of Computer Science

Carnegie Mellon University
Language Technologies Institute

Postodoctoral fellowship

Microsoft Research

Faculty Positions

İhsan Doğramacı Bilkent Üniversitesi 1984



Teaching Assistant

- Emine Ayşe Sunar
  PhD Candidate CS Program



ayse.sunar@sabanciuniv.edu

Office hour: TBD

Communication

- We will use SuCourse
- SuCourse will be updated regularly throughout the semester with lecture notes, reading list, assignments, and important deadlines.
- Discussions and announcements will be made on SuCourse.
- Use the forum for questions and answers

Asking Quesitons and Getting Help

- **Ask on SuCourse Forum**
  - Any questions that don’t reveal assignment solutions
- **Email the TA**
  - For private assignment questions
- **Email the instructor**
  - Missing exams or personal issues

About this course

- CS512 is a graduate-level introductory course aims at giving a broad overview of concepts and algorithms in machine learning.
- The class participants are from all different backgrounds
- Non-formal prerequisites:
  - Good programming skills
  - Basic algorithms, data structures knowledge
  - Basic probability and statistic
  - Basic linear algebra

Course goals

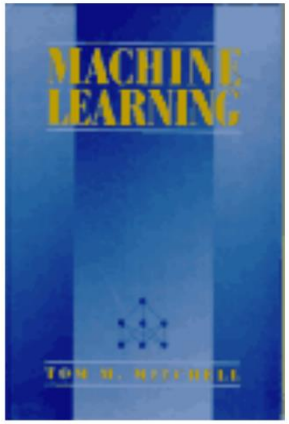
- By the end of CS512, you should be able to:
  - **Formulate** a real-world question as a machine learning problem and decide whether ML is appropriate.
  - **Explain and compare** core learning algorithms, including their assumptions and limitations.
  - **Build and evaluate** a model using suitable data, baselines, performance measures and correct evaluation pipelines.
  - **Interpret results** and identify when a model may fail on new data.

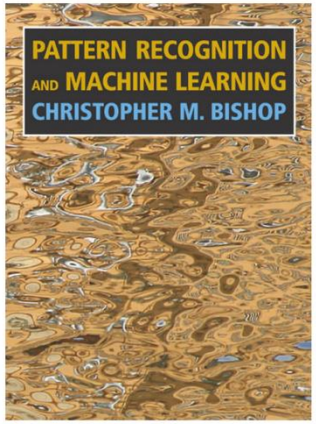
Tentative course topics

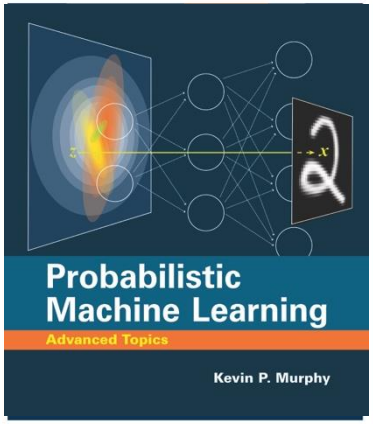
- This class should give you the basic foundation for applying machine learning
- **Model selection and evaluation**
- **Overfitting, Regularization, Optimization**
- **Supervised learning**
  - Naïve Bayes classifier
  - Linear models for regression and classification
  - Support vector machines
  - Decision trees
  - Ensemble models
  - Introduction to deep learning
- **Unsupervised learning**
  - Clustering
  - Dimensionality reduction
- **Additional topics (if time permits)**
  - Explainability and Interpretability
  - Active learning
  - Reinforcement learning

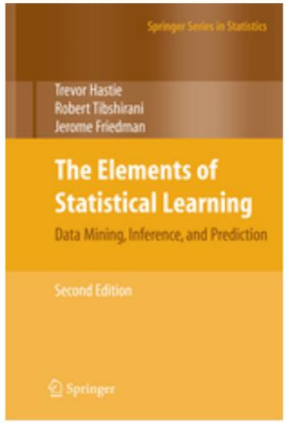
Textbooks and Other Resources

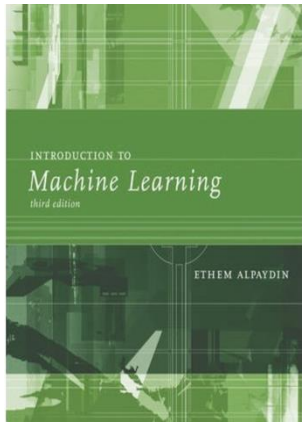
- No dedicated textbook.
- There will be required and optional readings, videos listed on SuCourse.
- Reference books:











- For more books please check the syllabus and for more reading list: https://machinelearningmastery.com/machine-learning-books/

Grading

- **Two midterms exam 30% each**
  - Physical
- **Three or four homework assignments 20%**
  - Both written and programming
- **Term project 20 %**
  - 3-5 people, will include a peer grade

Minimum Requirements

- One of the following conditions will result with an automatic F regardless of other grades:
  - Not submitting more than one assignment .
  - The average of the homework assignment scores is below 30.
  - Not submitting a project report.
  - Being absent in a project presentation.
  - Missing an exam without a medical report.

*** Not falling in one of the conditions does not guarantee passing the course, if your overall performance is poor, you will fail the course.

Homework Assignments

- Each one will involve both written and practical exercises

- You will use Python for programming.
- **Start early.**

Late day policy (Grace days☺)

- Total of **four free late (calendar) days** to use for homework assignments.
- Once these late days are exhausted, any assignments turned in late will be penalized 33% per late day.
- **No assignment will be accepted more than four days after its due date.**

Term Project

- **Groups and proposals**
- **Proposal Stage.**
  - **Write Up:** Maximum one-page proposal write up.
- **Progress Stage**
  - **Progress Report:** The progress stage will significantly affect the final grade of the project. You are expected to have results by the progress date.
  - **Progress Presentation:** There will be an in minute in class progress presentations on the work done so far.
- **Final Stage:**
  - **Final Report:** A final write up of the project. You should submit a pdf file electronically.
  - **Final Presentation:** Demo and presentation to the instructor

Details are in the syllabus, and we will make announcements on the dates.

Term project

- Any project of your choice can be proposed in machine learning
  - applying known methods to solve a problem
  - coming up with a new methodology
  - can be (and encouraged to be) related to your research
- Project must be something new (e.g., not work done last term or in another course)

Refer to the **syllabus** for detailed official course policies

To do for Week 1

- Enroll on SuCourse+
- Read the syllabus
- Read the required reading materials for this week
- Start talking to your classmates about project ideas

Raise your hand if you took a course on machine learning before

Raise your hand if you self-learned machine learning before

Raise your hand if you are a first year student

Raise your hand if you are a CS student.

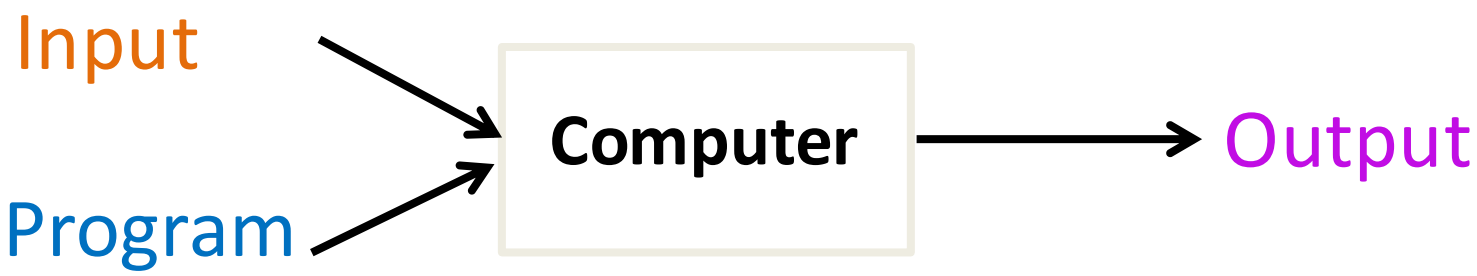
Today's Schedule

- Course logistics and outline
- An overview of machine learning

What is Machine Learning?

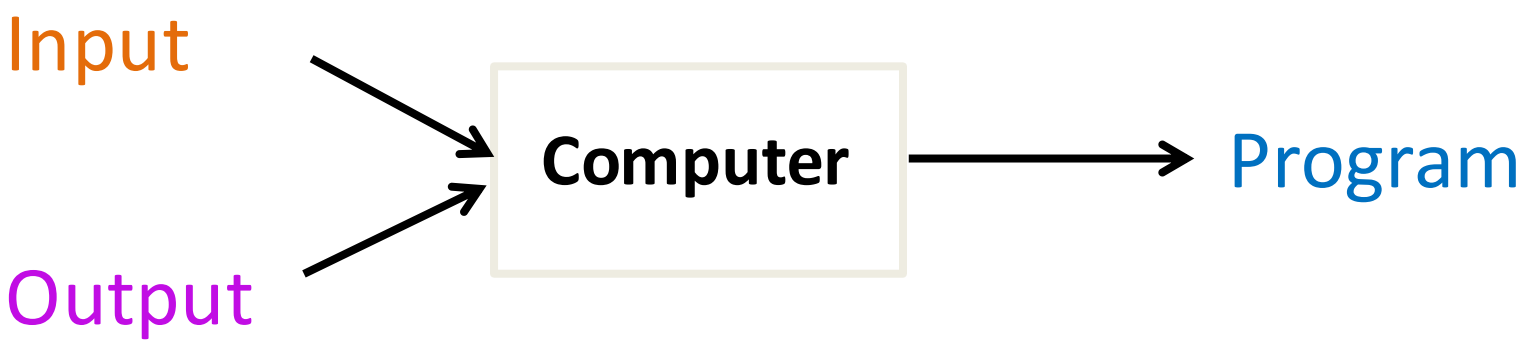
ML is the process of training a program, called a model, to make useful predictions or generate content (like text, images, or video) from data.

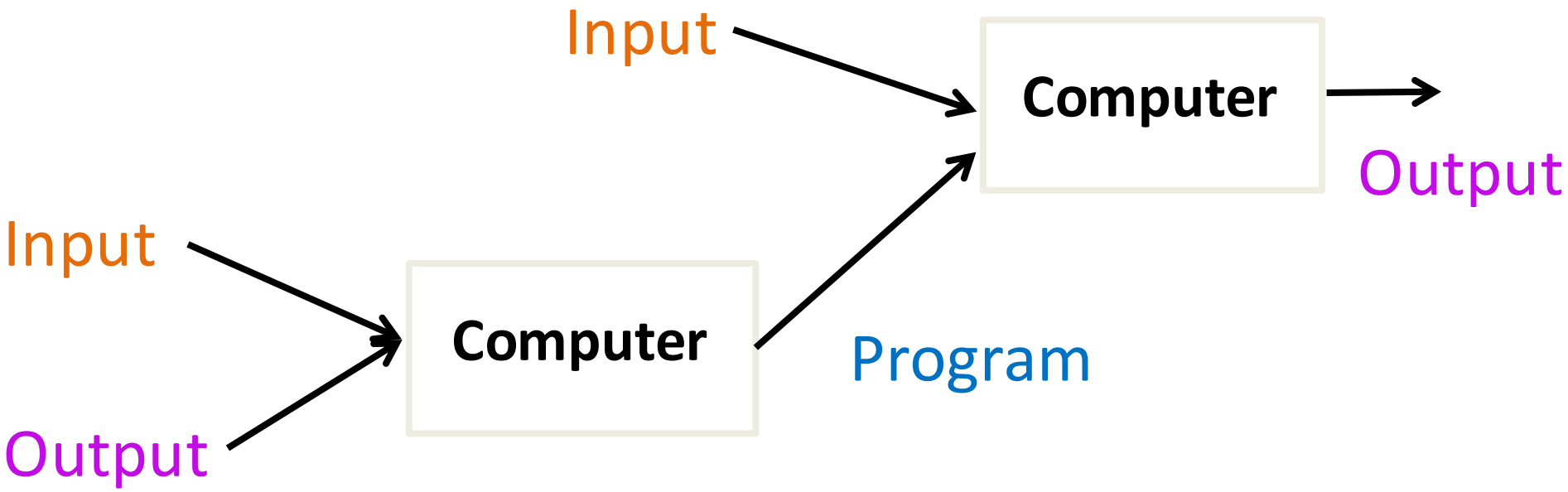
Traditional Programming



27

Machine Learning




Input
Computer
Output
Input
Computer
Program
Output

When learning is useful?

When learning is useful?

- Do we need machine learning for every problem?
  - To calculate the **area of a rectangle**?
  - To calculate the **interest earned on a bank deposit**?

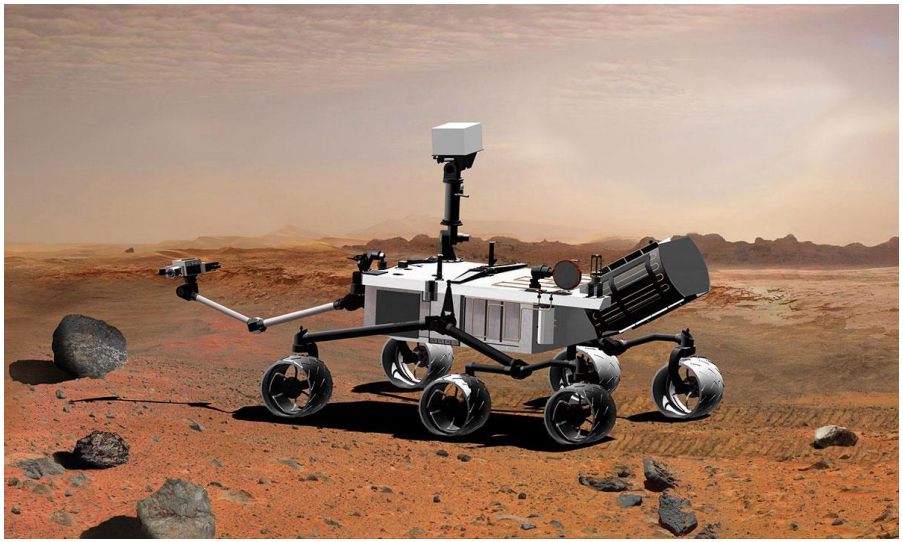
When ML is useful?

- Do we need machine learning for every problem?
  - To calculate the **area of a rectangle**?
  - To calculate the **interest earned on a bank deposit**?

  - **No.** When we know the formula or rules for solving a problem, we can program the solution directly.

Learning useful?

- **Problems where humans have limited expertise**
  Example: Navigating autonomously on the surface of Mars.



https://timeandnavigation.si.edu/multimedia-asset/curiosity-rover-on-mars

When is learning useful?

- **Tasks humans can perform but cannot explicitly describe the rules for**
- Example: Recognizing speech or identifying objects in images.

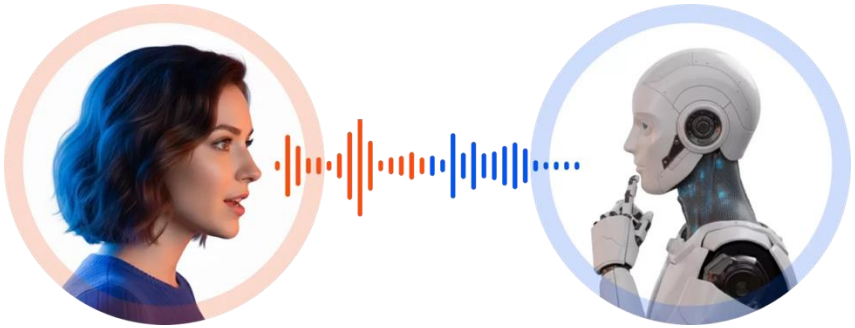
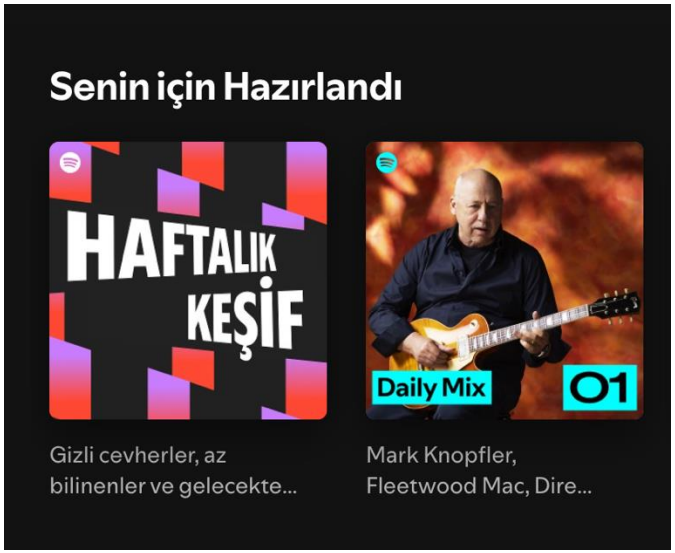


Figure source: https://www.kardome.com/resources/blog/difference-speech-and-voice-recognition/

When is learning useful?

- Solution needs to be adapted to particular cases
- Example: Personal recommendations



What is Machine Learning?

- **Arthur Samuel (1959) :**
  - Field of study that gives computers the ability to learn without being explicitly programmed.



- **Tom Mitchell (1998) :**
  - **Formally:** A computer program is said to learn from experience **E** with respect to some task T and some performance measure on T, as measured by P improves with experience **E.**
  - **Informally:** *Algorithms* that improve on some *task* with *experience*



Well-Posed Learning Problems

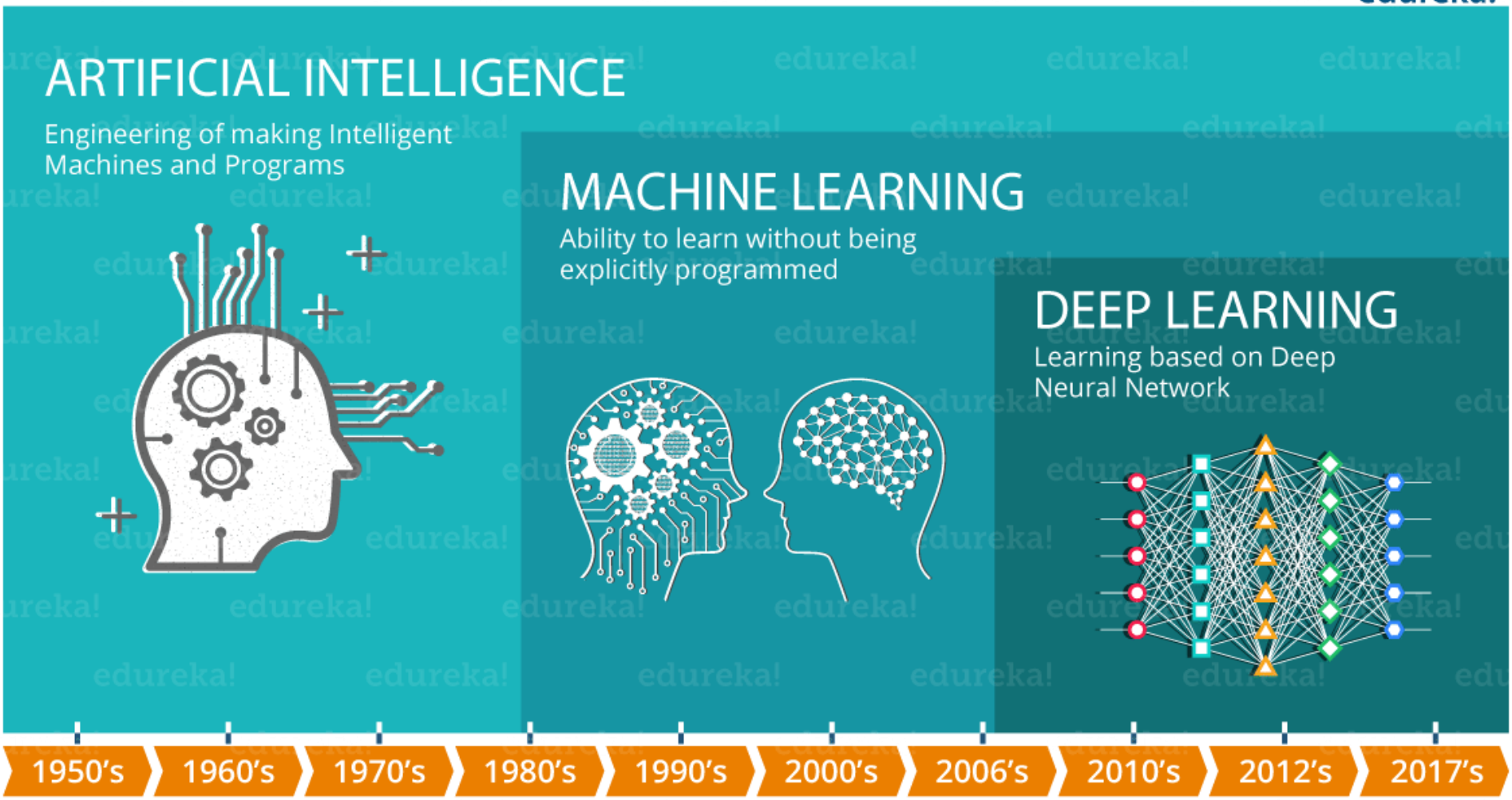
- **Three components** <*T,P,E*> **:**
  1. Task, *T*
  2. Performance measure, *P*
  3. Experience, *E*

- **Definition of learning:** A computer program learns if its performance at task *T*, as measured by *P*, improves with experience *E*.

Example Learning Problem

Learning to respond to voice commands:

1. Task, *T*:
   predicting action from speech
2. Performance measure, *P*:
   percent of correct actions taken in user pilot study
3. Experience, *E*:
   examples of (speech, action) pairs



Source: coursera

Experience = data

- Humans learn to improve their skills through practice

- The machine learning goal is to write algorithms that will improve with **experience**
  - Experience is encoded as **examples, or data**
  - Autonomous learning must gain from experience

- Where does training data come from?
  - Human annotation
  - Automatic collection

Objectives of Machine Learning

- **Algorithms:**
  - design of efficient, accurate, and general learning algorithms to deal with large-scale problems
  - make accurate predictions (unseen examples)
  - handle a variety of different learning problems

- **Theoretical questions:**
  - What can be learned? Under what conditions?
  - What learning guarantees can be given?
  - What can we say about the inherent ease or difficulty of learning problems?

Early Machine Learning: 1950s–1970s

- Computers began learning from examples and experience.
  - **1950s:** Samuel's checkers program and Rosenblatt's perceptron.
  - **1970s:** Systems learned concepts and symbolic rules, including rules for scientific discovery.

- **Key idea:** Program the learning process, rather than every decision.

Theory and Statistical Learning: 1980s–2000s

- Learning theory studied when models can generalize.
- Backpropagation helped multilayer neural networks learn representations.
- Decision trees, probabilistic models, SVMs, and ensembles became widely used.
- Reinforcement learning developed methods for learning through interaction and rewards.

- **Key idea:** Build models that work well on unseen data.

Deep Learning and Scale: 2010s

- Larger datasets and faster computation enabled more powerful neural networks.
- **2012:** AlexNet's ImageNet success accelerated deep learning adoption.
- **2017:** Transformers introduced a powerful attention-based architecture.
- Self-supervised learning enabled training using targets derived from the data itself.

- **Key idea:** Learn useful representations from large amounts of data.

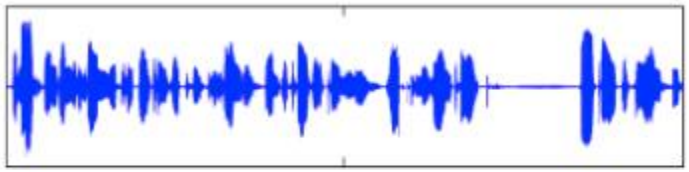
Growth of Machine Learning

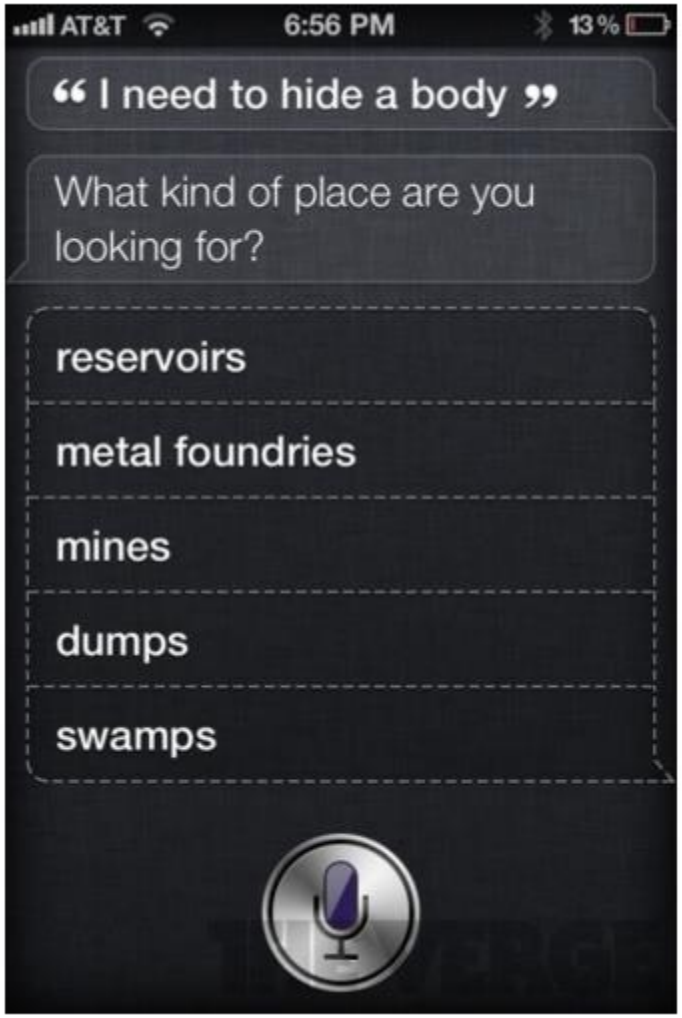
- This trend is accelerating
  - Big data
  - Improved machine learning algorithms
  - Faster computers
  - Good open-source software
  - Talent attraction

From Specialized Models to Foundation Models

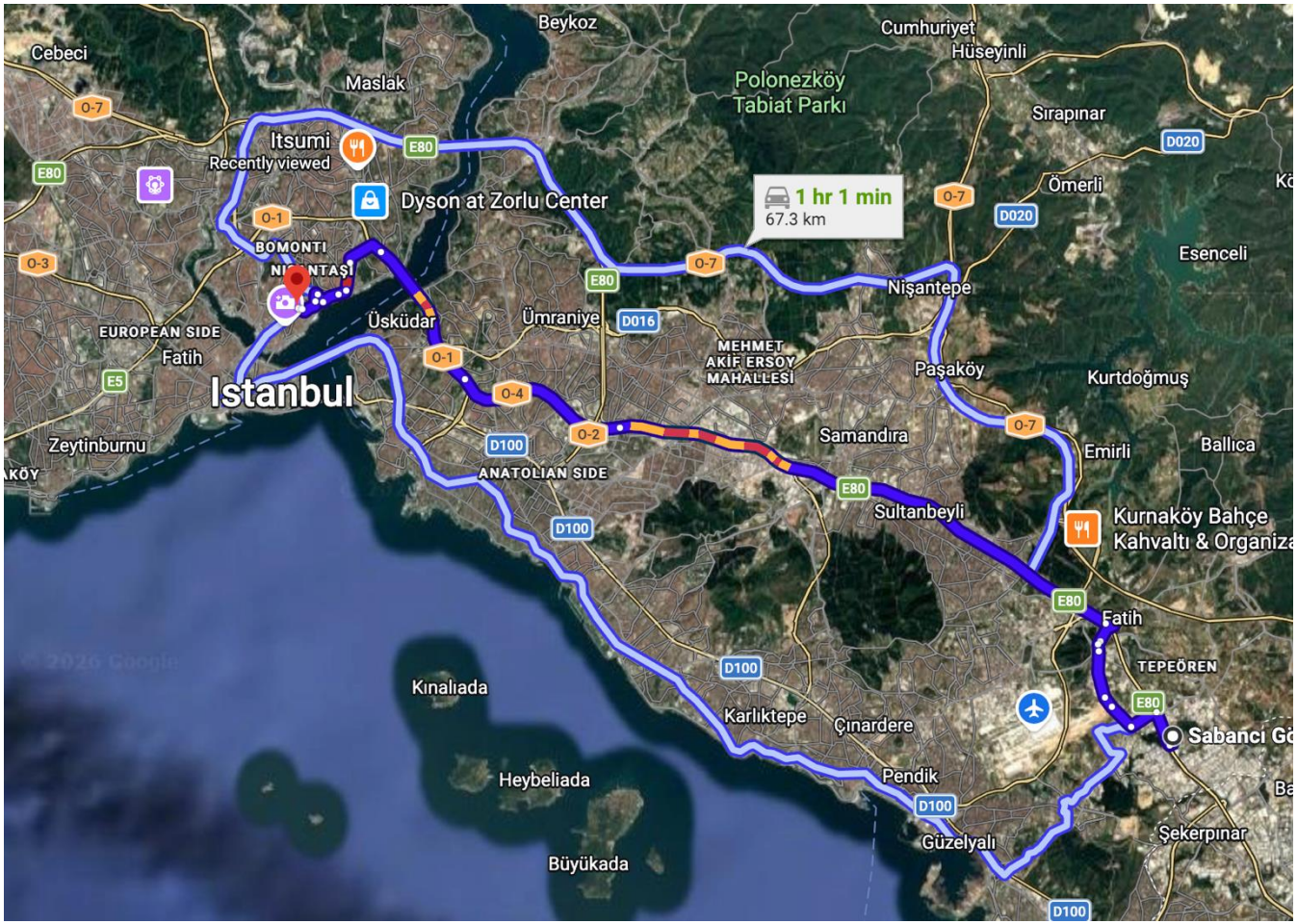
- Large pretrained models can support many tasks.
- Models can be adapted through fine-tuning or prompting.
- Reliability, generalization, and careful evaluation remain essential.
- **2020 - GPT-3:** Demonstrated substantial few-shot task performance from a large autoregressive language model.
- Modern ML builds on decades of ideas - and different approaches continue to coexist.

Speech Recognition



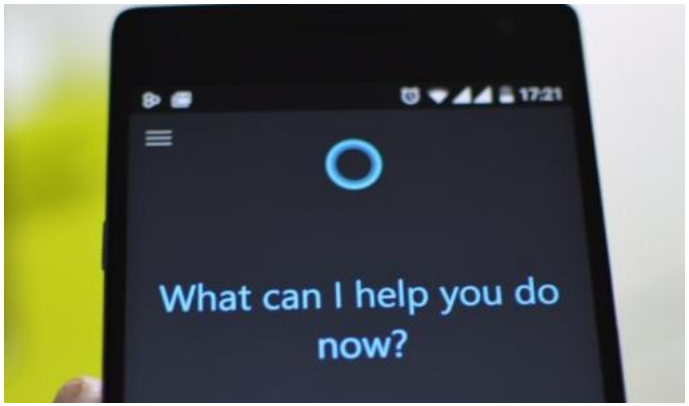


Traffic Prediction



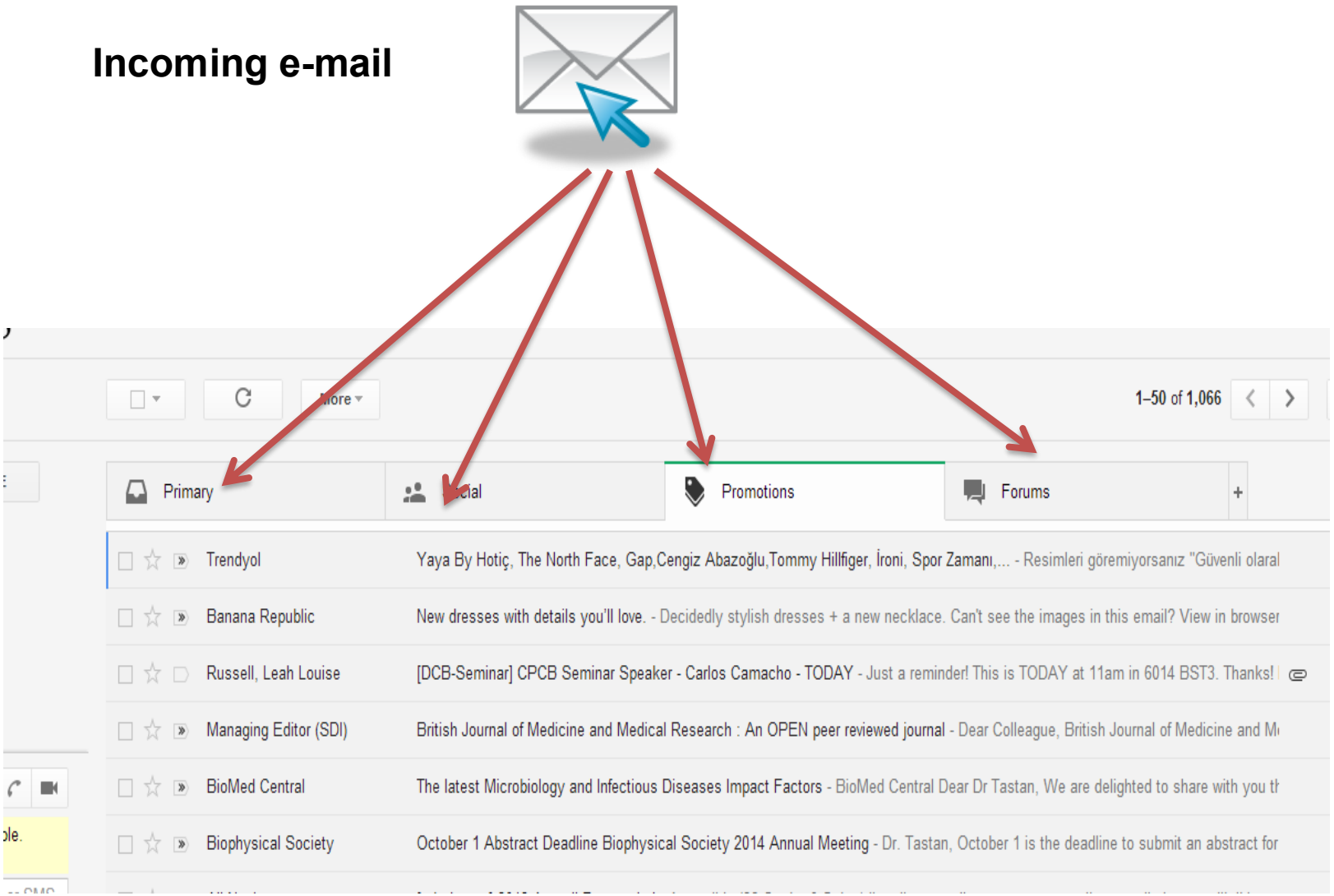
Everyday Technology



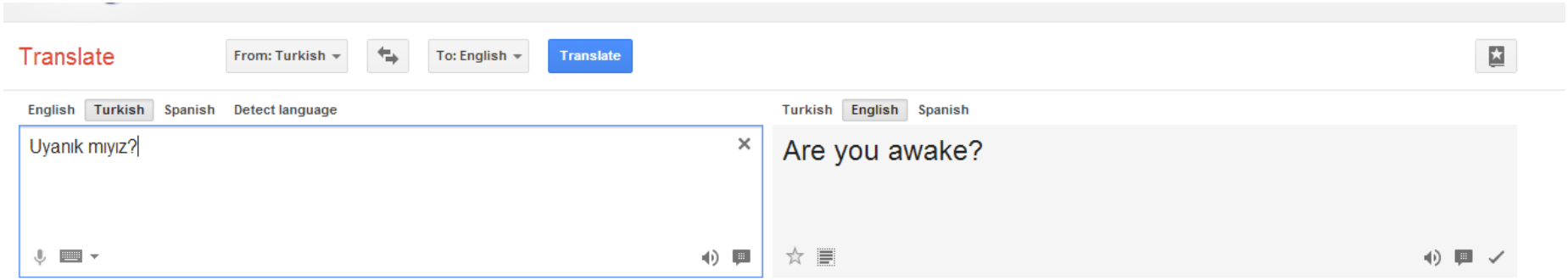


Text Classification

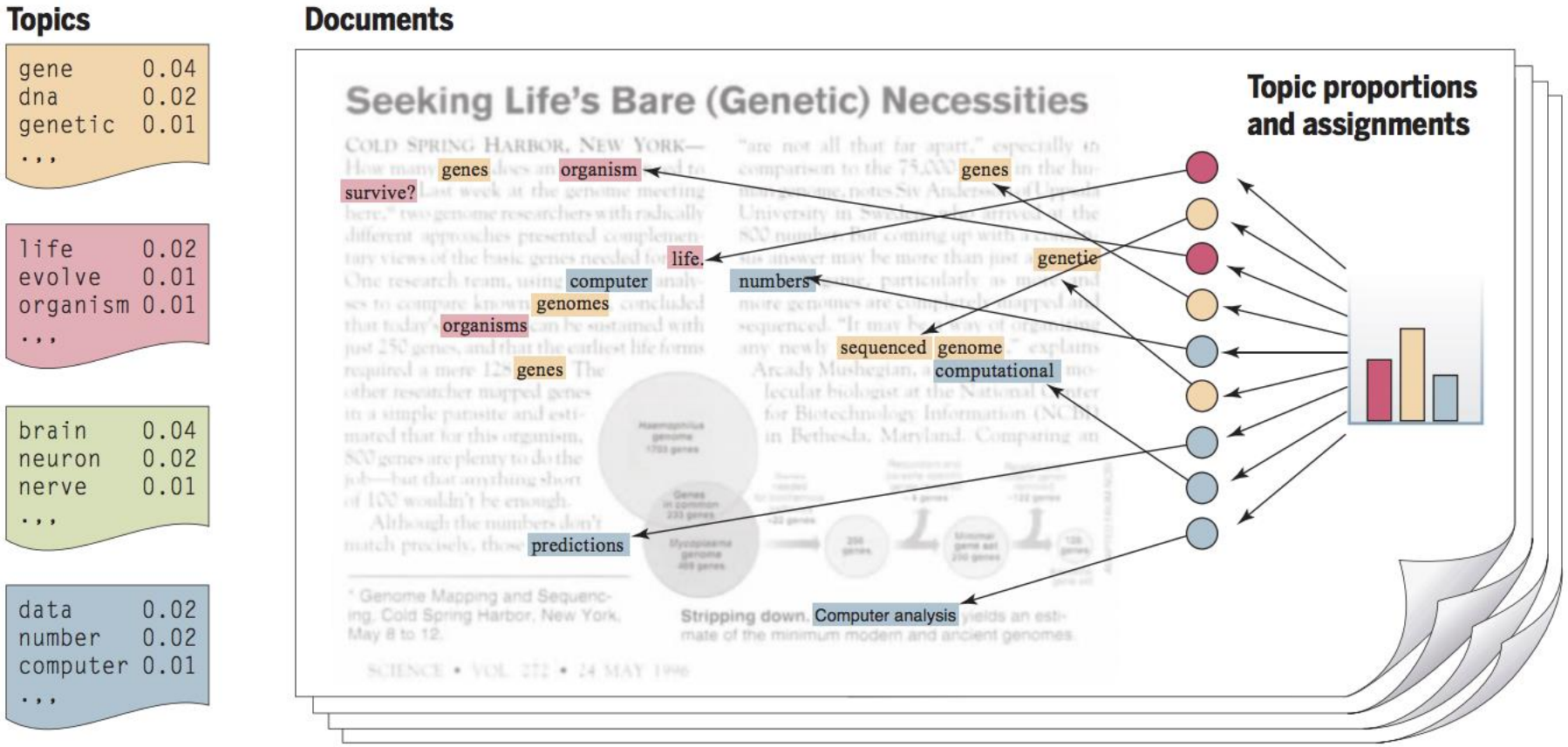
Incoming e-mail



Statistical Machine Translation

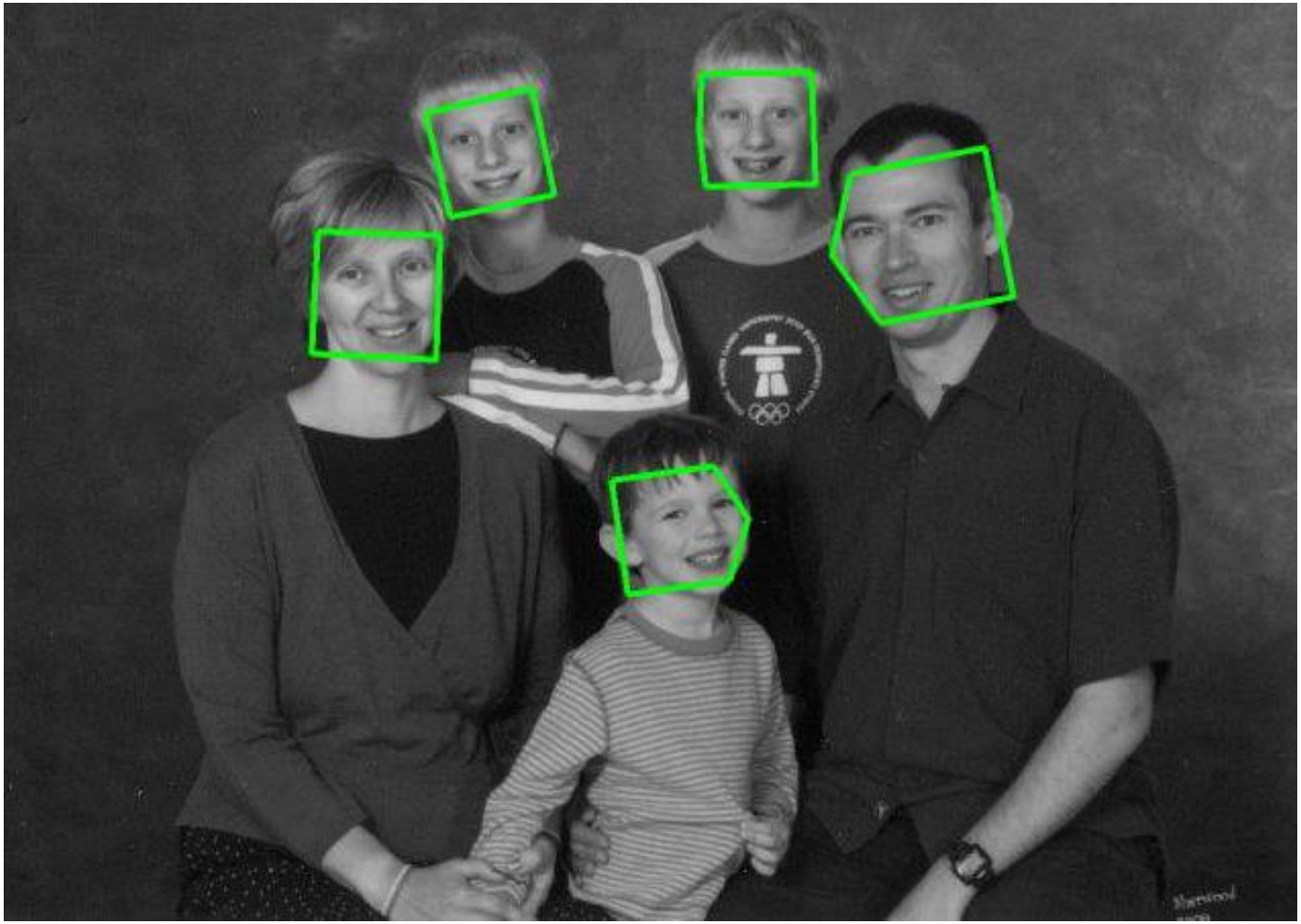


Topic Modeling

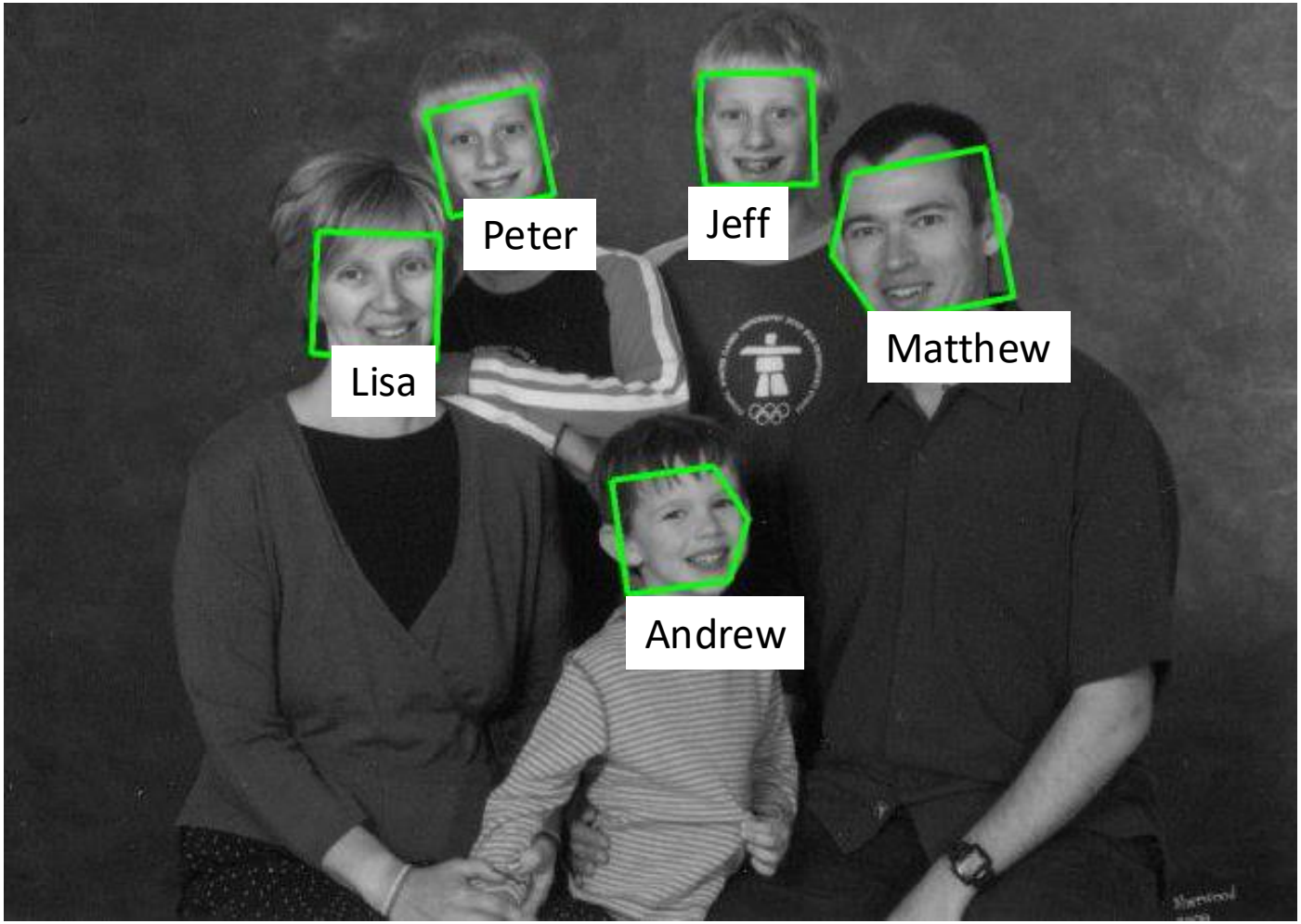


D. Blei, Commun. ACM 55, 77–84 (2012).

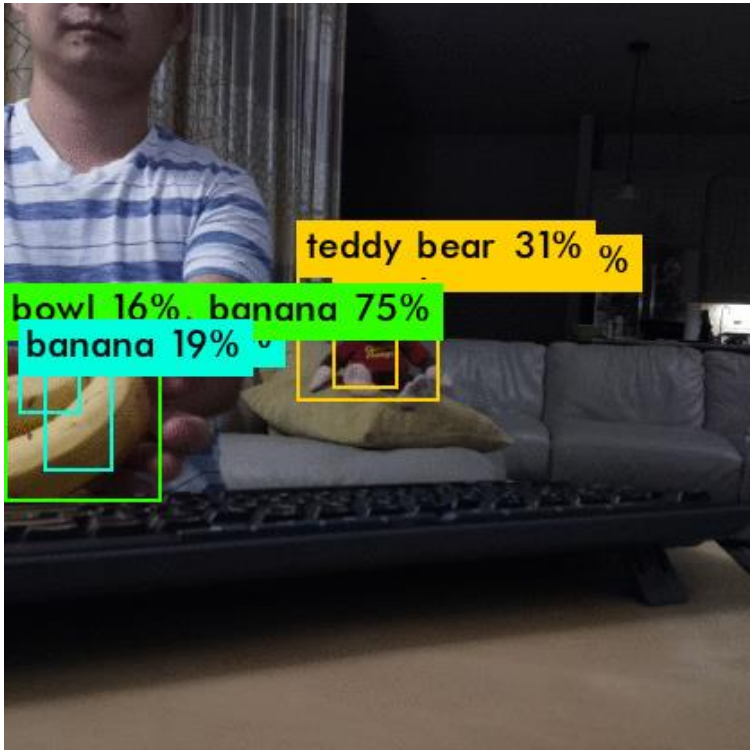
Detecting Faces



Recognizing Faces



Object Recognition



Semantic Segmentation



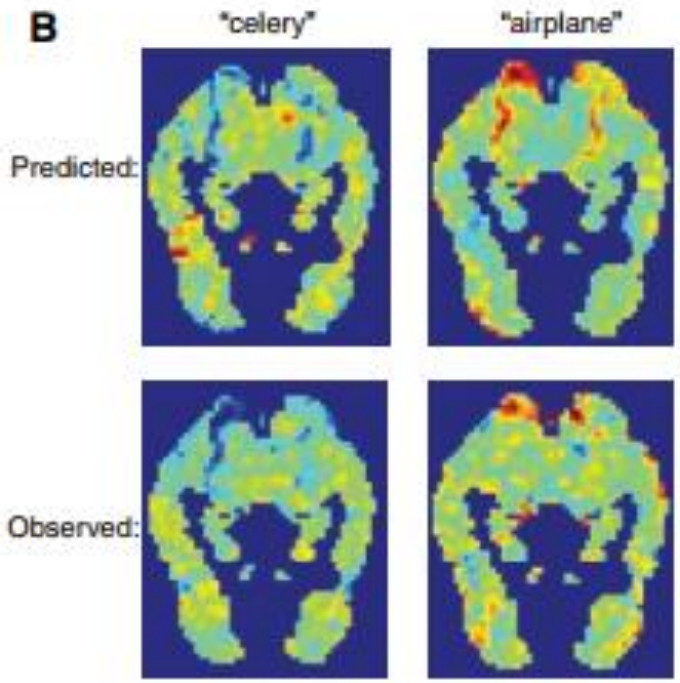
Generative Models

Prompt: A stylish woman walks down a Tokyo street filled with warm glowing neon and animated city signage. She wears a black leather jacket, a long red dress, and black boots, and carries a black purse. She wears sunglasses and red lipstick. She walks confidently and casually. The street is damp and reflective, creating a mirror effect of the colorful lights. Many pedestrians walk about.



Source: https://openai.com/sora

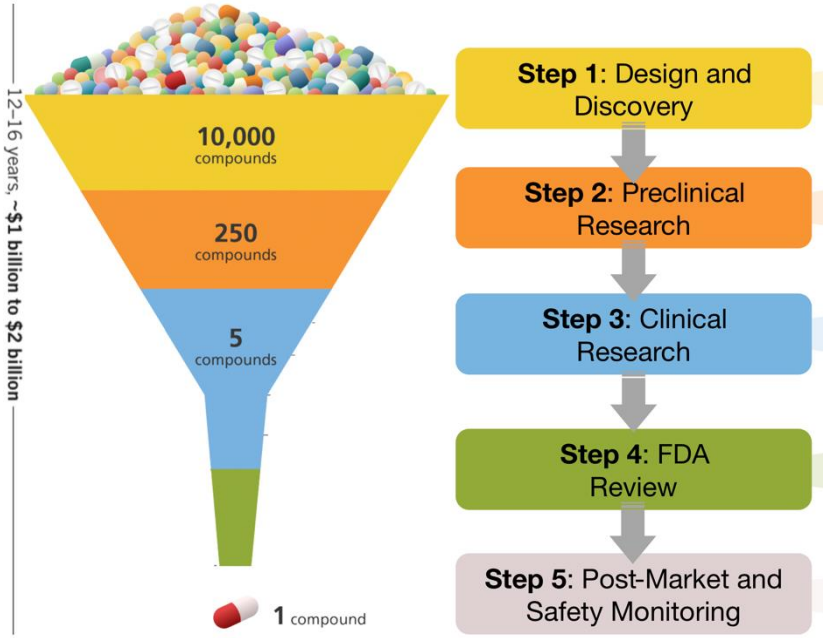
Analyzing fMRI Data



Mitchell et.al, Science, 2009.

- Predict activation patterns for nouns

Drug Discovery



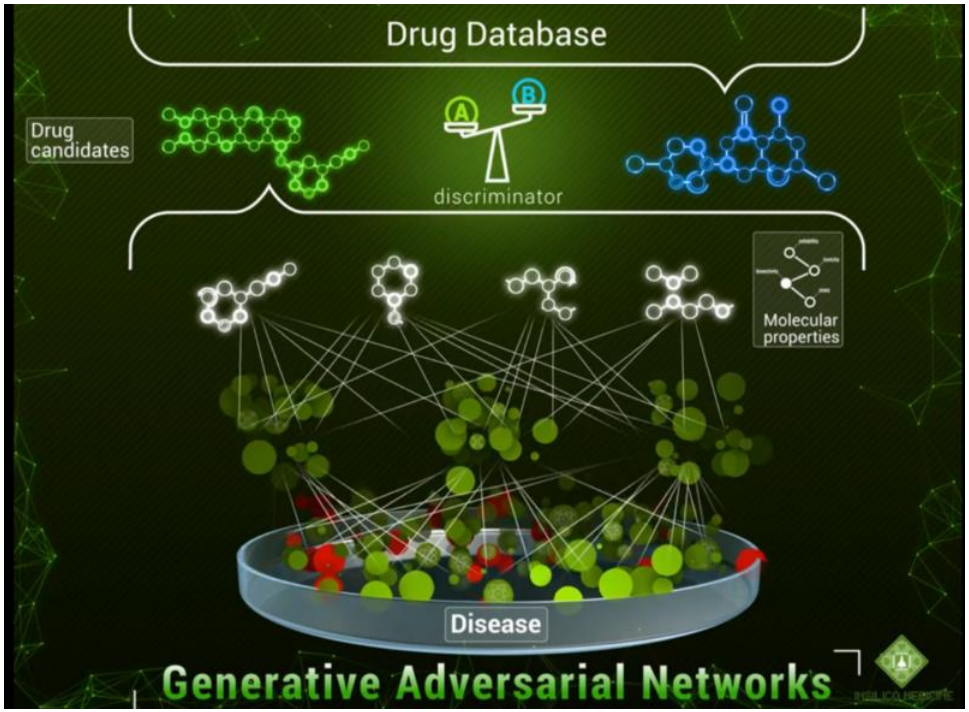


Figure source: Marinka Zitnik

https://neuromation.io/en/creating-molecules-scratch-drug-discovery-generative-adversarial-networks/

Prediction in the Intensive Care Units

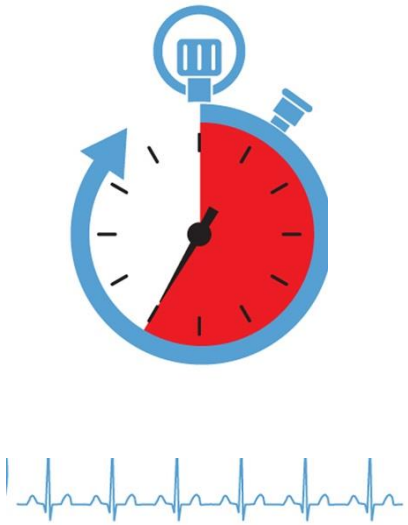
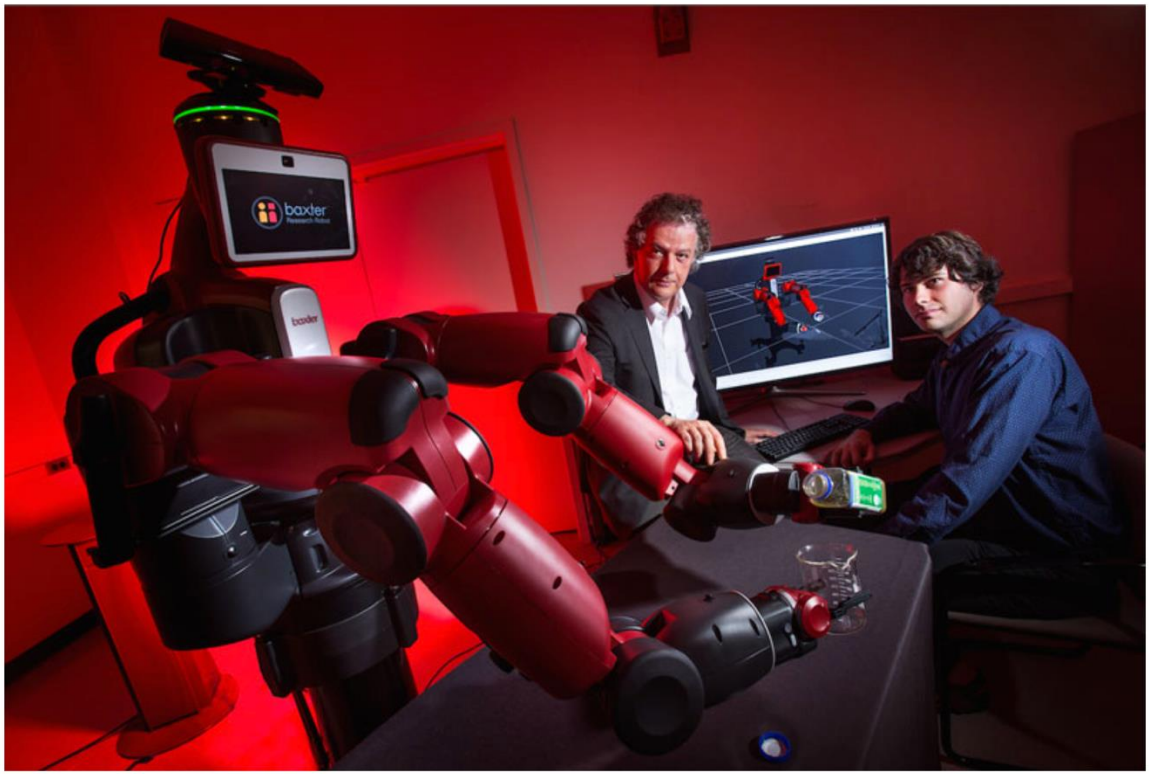




Figure source:McKibillo

Teaching a robot to ‘cook’ by showing it YouTube videos of cooking shows



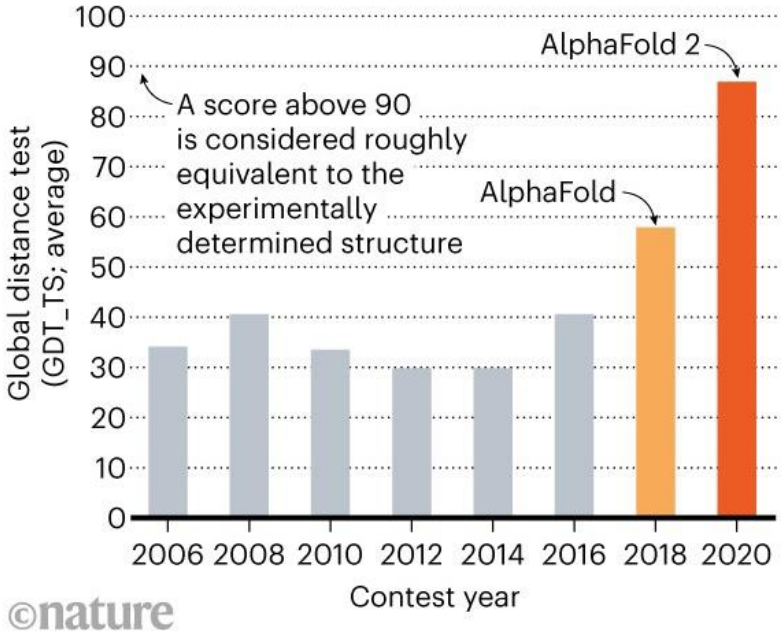
University of Maryland computer scientist Yiannis Aloimonos (center) is developing robotic systems able to visually recognize objects and generate new behavior based on those observations.

Photo credit: John T. Consoli

Bolder Tasks!

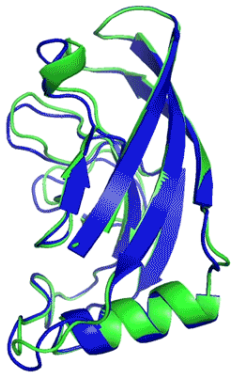


Protein Structure Prediction





T1037 / 6vr4
90.7 GDT
(RNA polymerase domain)



T1049 / 6y4f
93.3 GDT
(adhesin tip)

Experimental result
Computational prediction

Supervised Binary Trash Classifier

Suppose the environmental task force ask you to build a robot for collecting trash in the rivers.



- You are tasked with building a classifier that detects whether an object is a piece of trash (+) or not a piece of trash (-)
- The robot can detect an object's color, sound, and weight
- You manually annotate the following dataset based on objects you find

trash?	color	sound	weight
+	green	crinkly	high
-	brown	crinkly	low
-	grey	none	high
+	clear	none	low
-	green	none	low

Slides from CMU 10601

Supervised Binary Classification

- Def: an **example** contains a label (aka. class) and features (aka. point or attributes)
- Def: a **labeled dataset** consists of rows, where each row is an example
- Def: an **unlabeled dataset** only has features

One example:

label	features		
trash?	**color**	**sound**	**weight**
-	brown	none	high

Labeled Dataset:

	label	features		
index	**trash?**	**color**	**sound**	**weight**
1	-	brown	none	high
2	+	clear	crinkly	low
3	-	brown	none	low

Unlabeled Dataset:

	features		
index	**color**	**sound**	**weight**
1	brown	none	high
2	clear	crinkly	low
3	brown	none	low

Training

- Def: an **example** contains a label (aka. class) and features (aka. point or attributes)
- Def: a **labeled dataset** consists of rows, where each row is an example
- Def: an **unlabeled dataset** only has features

- Def: a **classifier** is a function that takes in features and predicts a label
- Def: a **training dataset** is a labeled dataset used to **learn** a classifier
- Def: a **test dataset** is a labeled dataset used to **evaluate** a classifier



Training Dataset:

	label	features		
index	**trash?**	**color**	**sound**	**weight**
1	+	green	crinkly	high
2	-	brown	crinkly	low
3	-	grey	none	high
4	+	clear	none	low
5	-	green	none	low

Test Dataset:

	label	features		
index	**trash?**	**color**	**sound**	**weight**
1	-	brown	none	high
2	+	clear	crinkly	low
3	-	brown	none	low

Prediction

- Def: **predictions** are the output of a trained classifier
- Def: **error rate** is the proportion of examples on which we predicted the wrong label

- Def: a **classifier** is a function that takes in features and predicts a label
- Def: a **training dataset** is a labeled dataset used to **learn** a classifier
- Def: a **test dataset** is a labeled dataset used to **evaluate** a classifier

Classifier
features → label

Test Predictions:

predictions

index	trash?
1	+
2	+
3	-

(Unlabeled) Test Dataset:

features

index	color	sound	weight
1	brown	none	high
2	clear	crinkly	low
3	brown	none	low

Evaluation

- Def: **predictions** are the output of a trained classifier
- Def: **error rate** is the proportion of examples on which we predicted the wrong label

- Def: a **classifier** is a function that takes in features and predicts a label
- Def: a **training dataset** is a labeled dataset used to **learn** a classifier
- Def: a **test dataset** is a labeled dataset used to **evaluate** a classifier

error rate = 1/3

Test Predictions:

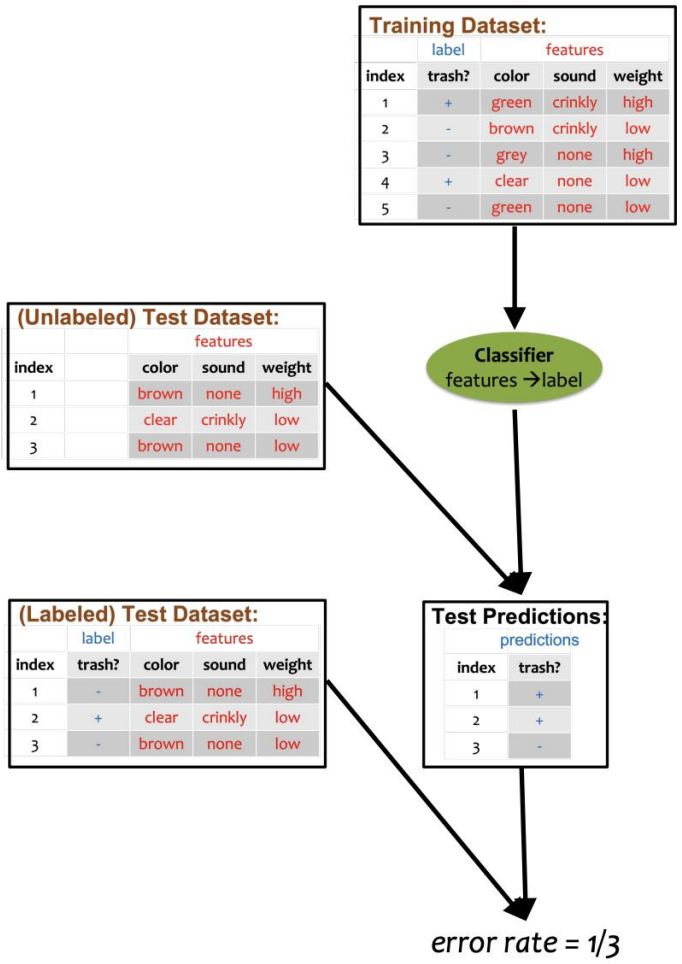
	predictions
index	**trash?**
1	+
2	+
3	-

(Labeled) Test Dataset:

	label	features		
index	**trash?**	**color**	**sound**	**weight**
1	-	brown	none	high
2	+	clear	crinkly	low
3	-	brown	none	low

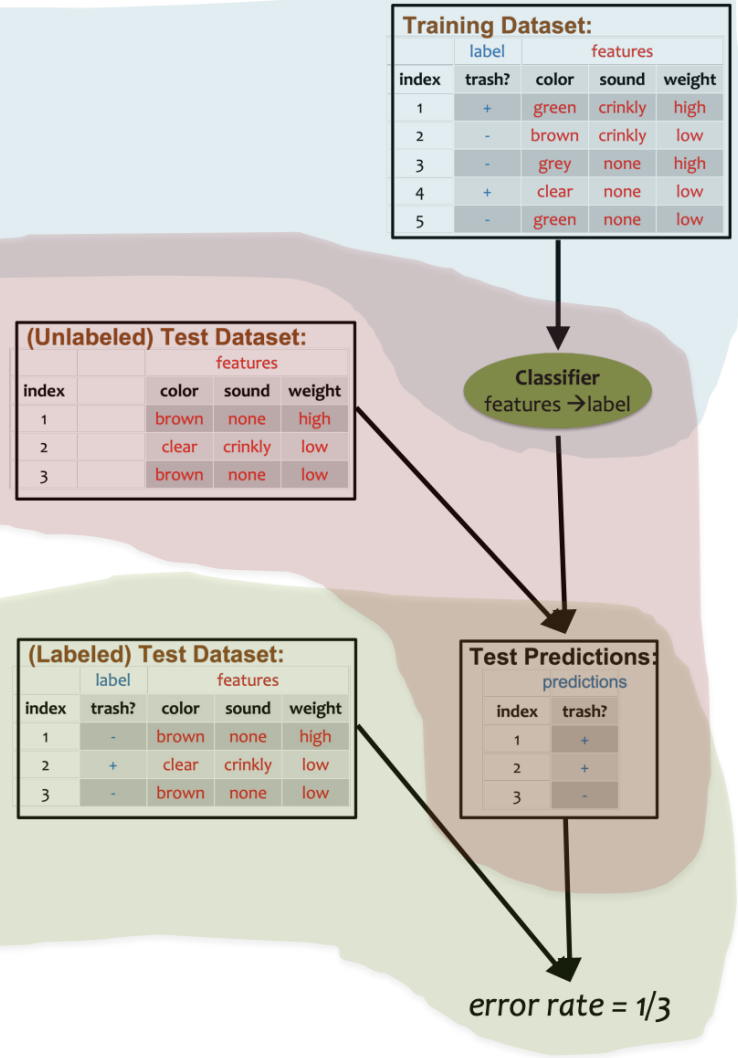
Supervised Binary Classification

- Step 1: training
  - *Given*: labeled **training dataset**
  - *Goal*: learn a **classifier** from the training dataset
- Step 2: prediction
  - *Given*: unlabeled **test dataset**
    : learned classifier
  - *Goal:* **predict** a label for each instance
- Step 3: evaluation
  - *Given*: **predictions** from Step II
    : labeled **test dataset**
  - *Goal:* compute the **test error rate** (i.e. error rate on the test dataset)



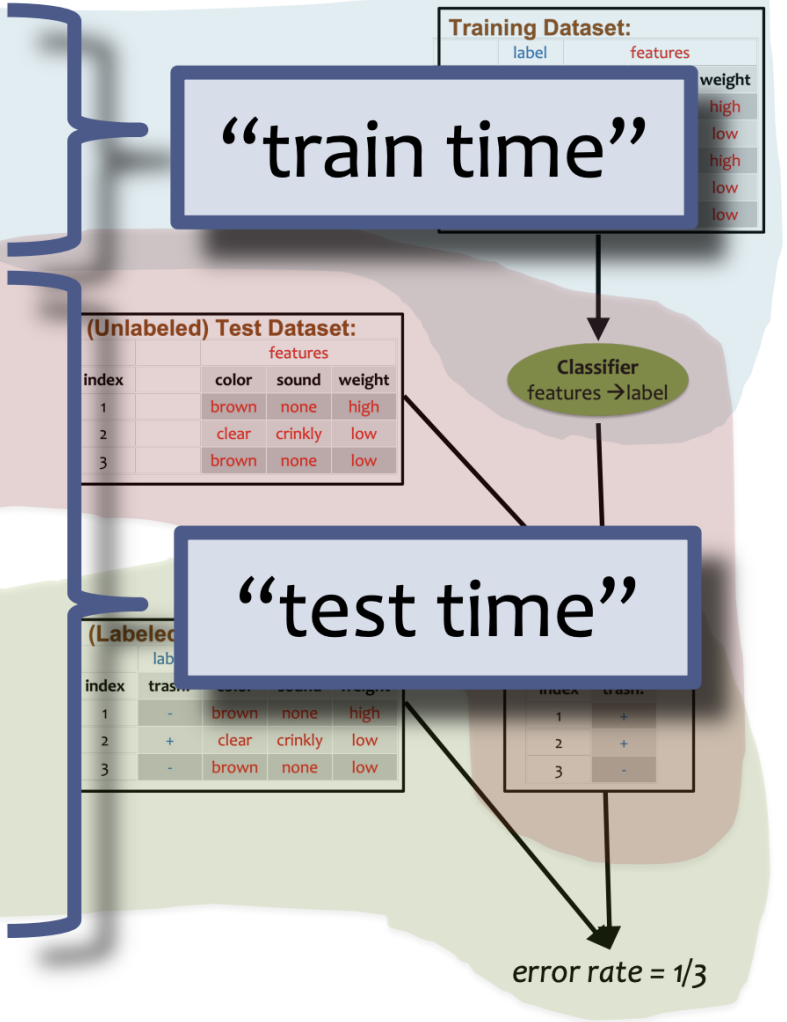
Supervised Binary Classification

- Step 1: training
  - *Given*: labeled **training dataset**
  - *Goal*: learn a **classifier** from the training dataset
- Step 2: prediction
  - *Given*: unlabeled **test dataset**
    : learned classifier
  - *Goal:* **predict** a label for each instance
- Step 3: evaluation
  - *Given*: **predictions** from Step II
    : labeled **test dataset**
  - *Goal:* compute the **test error rate** (i.e. error rate on the test dataset)



Supervised Binary Classification

- Step 1: training
  - *Given*: labeled **training dataset**
  - *Goal*: learn a **classifier** from the training dataset
- Step 2: prediction
  - *Given*: unlabeled **test dataset**
    : learned classifier
  - *Goal:* **predict** a label for each instance
- Step 3: evaluation
  - *Given*: **predictions** from Phase II
    : labeled **test dataset**
  - *Goal:* compute the **test error rate** (i.e. error rate on the test dataset)



Random Classifier

The **random classifier** takes in the features and always predicts a random label.

…this is a terrible idea. It completely **ignores the training data**!

Classifier
features →random!

Training Dataset:

	label	features		
index	trash?	color	sound	weight
1	+	green	crinkly	high
2	-	brown	crinkly	low
3	-	grey	none	high
4	+	clear	none	low
5	-	green	none	low

Test Predictions:

	predictions
index	trash?
1	-
2	-
3	+

error rate = 2/3

Test Dataset:

	label	features		
index	trash?	color	sound	weight
1	-	brown	none	high
2	+	clear	crinkly	low
3	-	brown	none	low

Random Classifier

The **random classifier** takes in the features and always predicts a random label.

…this is a terrible idea. It completely **ignores the training data**!

Classifier
features →random!

Training Dataset:

	label	features		
index	trash?	color	sound	weight
1	+	green	crinkly	high
2	-	brown	crinkly	low
3	-	grey	none	high
4	+	clear	none	low
5	-	green	none	low

Test Predictions:

	predictions
index	trash?
1	+
2	-
3	+

error rate = 3/3

Test Dataset:

	label	features		
index	trash?	color	sound	weight
1	-	brown	none	high
2	+	clear	crinkly	low
3	-	brown	none	low

Majority Vote Classifier

The **majority vote classifier** takes in the features and always predicts the **most common label** in the training dataset.

…this is still a pretty bad idea. It completely **ignores the features**!

Classifier
features →always predict "-"

Training Dataset:

	label	features		
index	trash?	color	sound	weight
1	+	green	crinkly	high
2	-	brown	crinkly	low
3	-	grey	none	high
4	+	clear	none	low
5	-	green	none	low

Test Predictions:

	predictions
index	trash?
1	-
2	-
3	-

error rate = 1/3

Test Dataset:

	label	features		
index	trash?	color	sound	weight
1	-	brown	none	high
2	+	clear	crinkly	low
3	-	brown	none	low

Majority Vote Classifier

The **majority vote classifier** takes in the features and always predicts the **most common label** in the training dataset.

…this is still a pretty bad idea. It completely **ignores the features**!

Classifier
features →always predict "-"

The majority vote classifier even ignores the features if it's making predictions on the training dataset!

error rate = 2/5

Train Predictions:

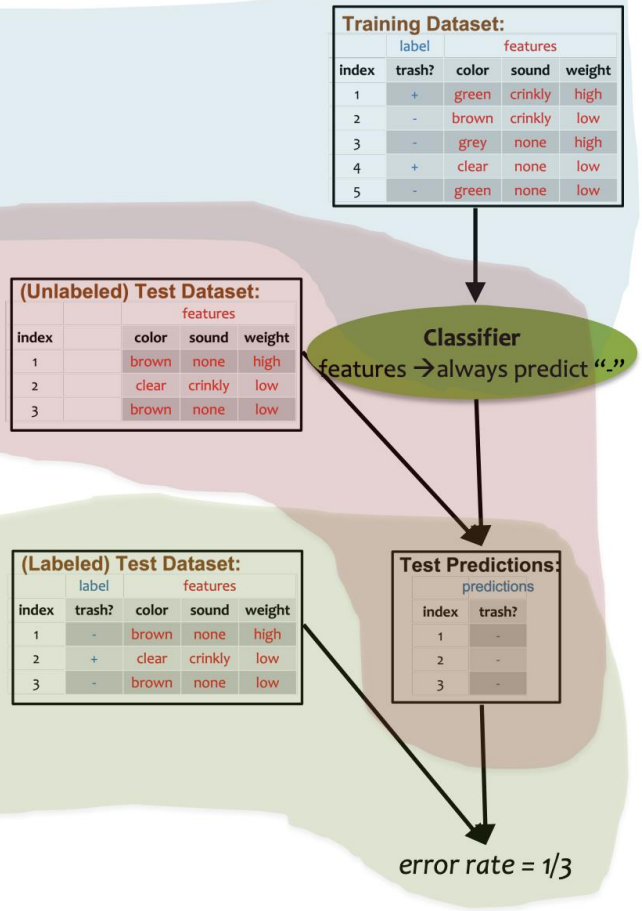
index	predictions: trash?
1	-
2	-
3	-
4	-
5	-

Training Dataset:

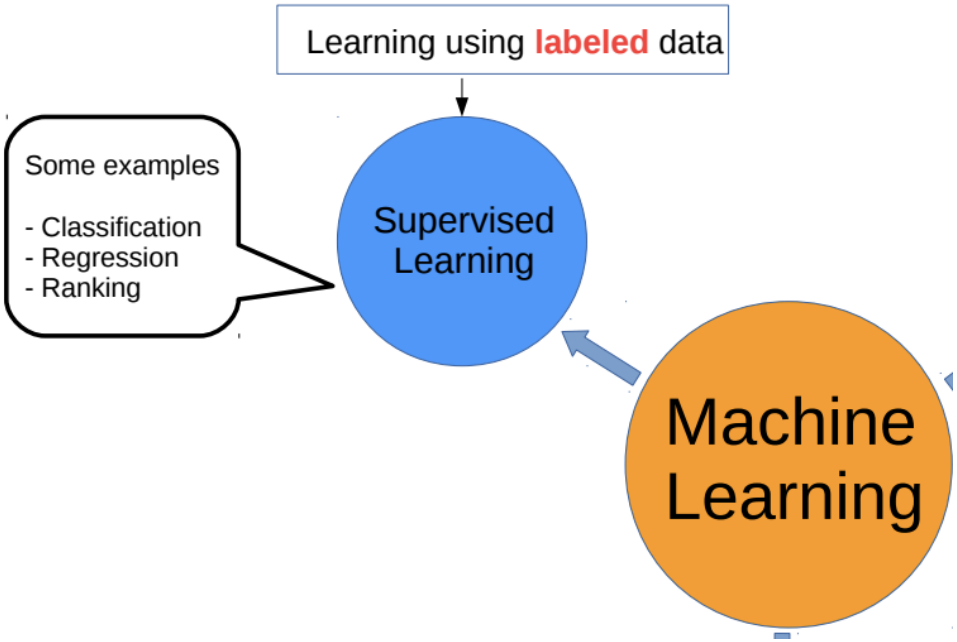
index	label: trash?	features: color	sound	weight
1	+	green	crinkly	high
2	-	brown	crinkly	low
3	-	grey	none	high
4	+	clear	none	low
5	-	green	none	low

Majority Vote Classifier

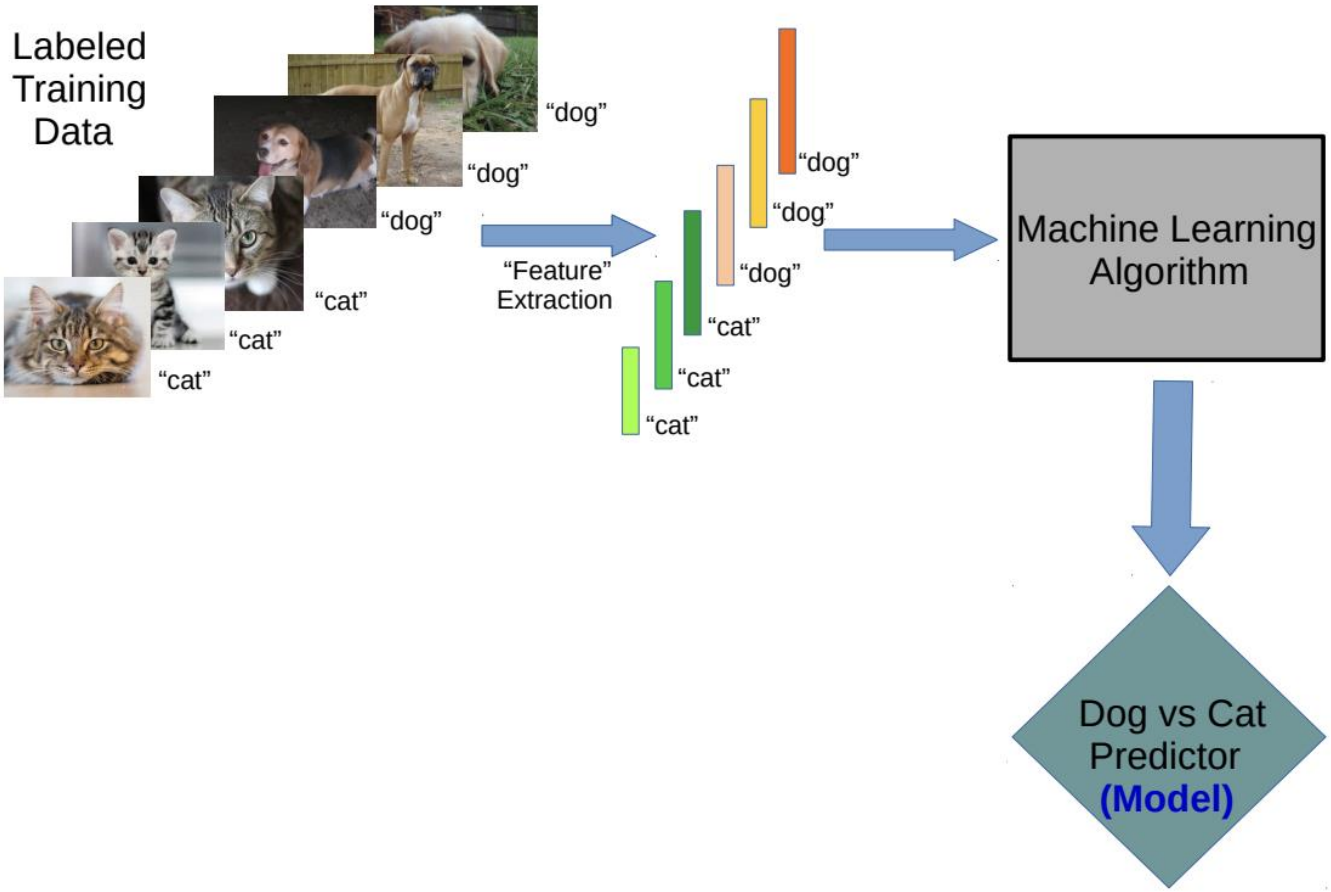
- Step 1: training
  - *Given*: labeled **training dataset**
  - *Goal*: learn a **classifier** from the training dataset
- Step 2: prediction
  - *Given*: unlabeled **test dataset**
    : learned classifier
  - *Goal:* **predict** a label for each instance
- Step 3: evaluation
  - *Given*: **predictions** from Step II
    : labeled **test dataset**
  - *Goal:* compute the **test error rate** (i.e. error rate on the test dataset)



A loose taxonomy of ML

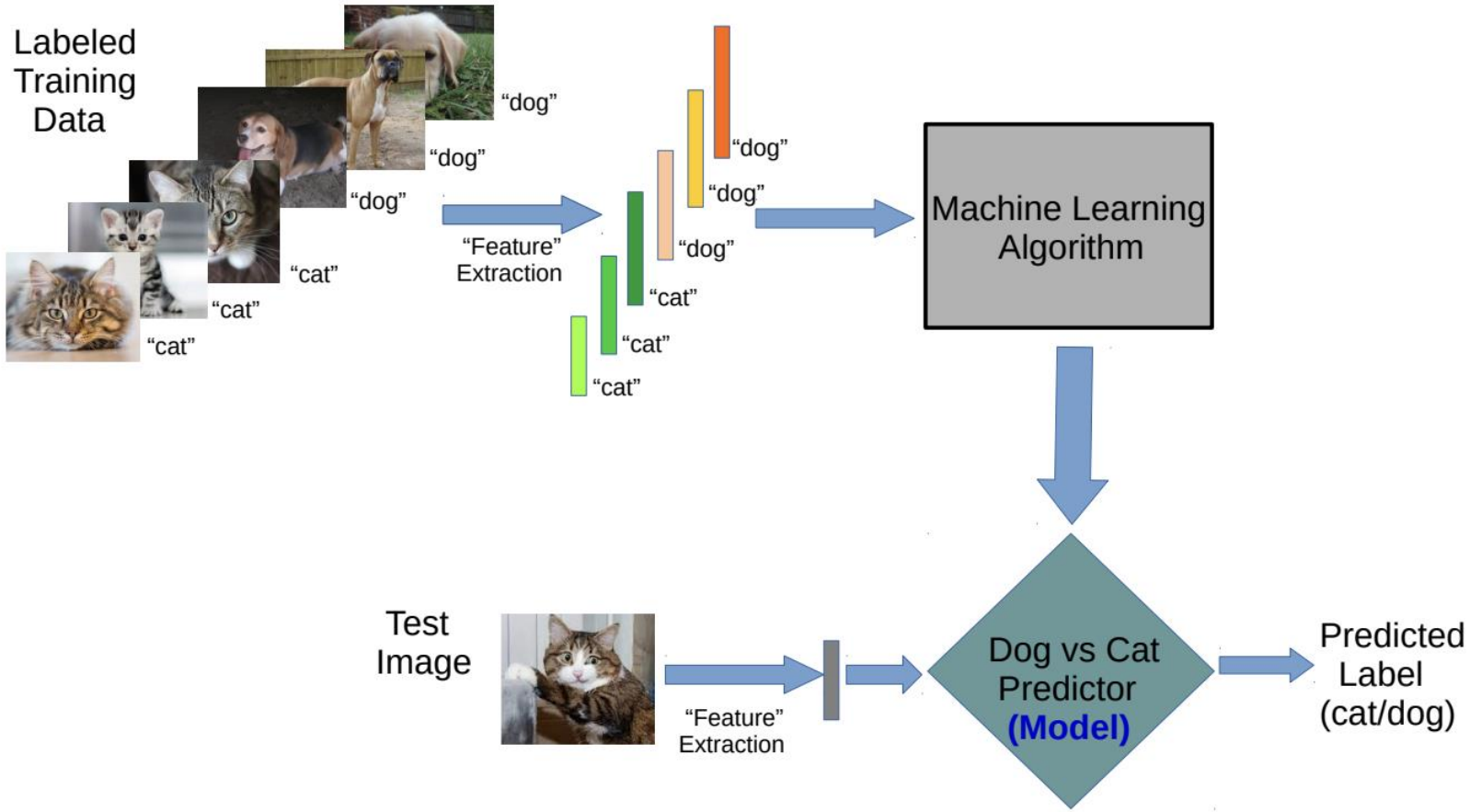


A Typical Supervised Learning Workflow (for Classification)



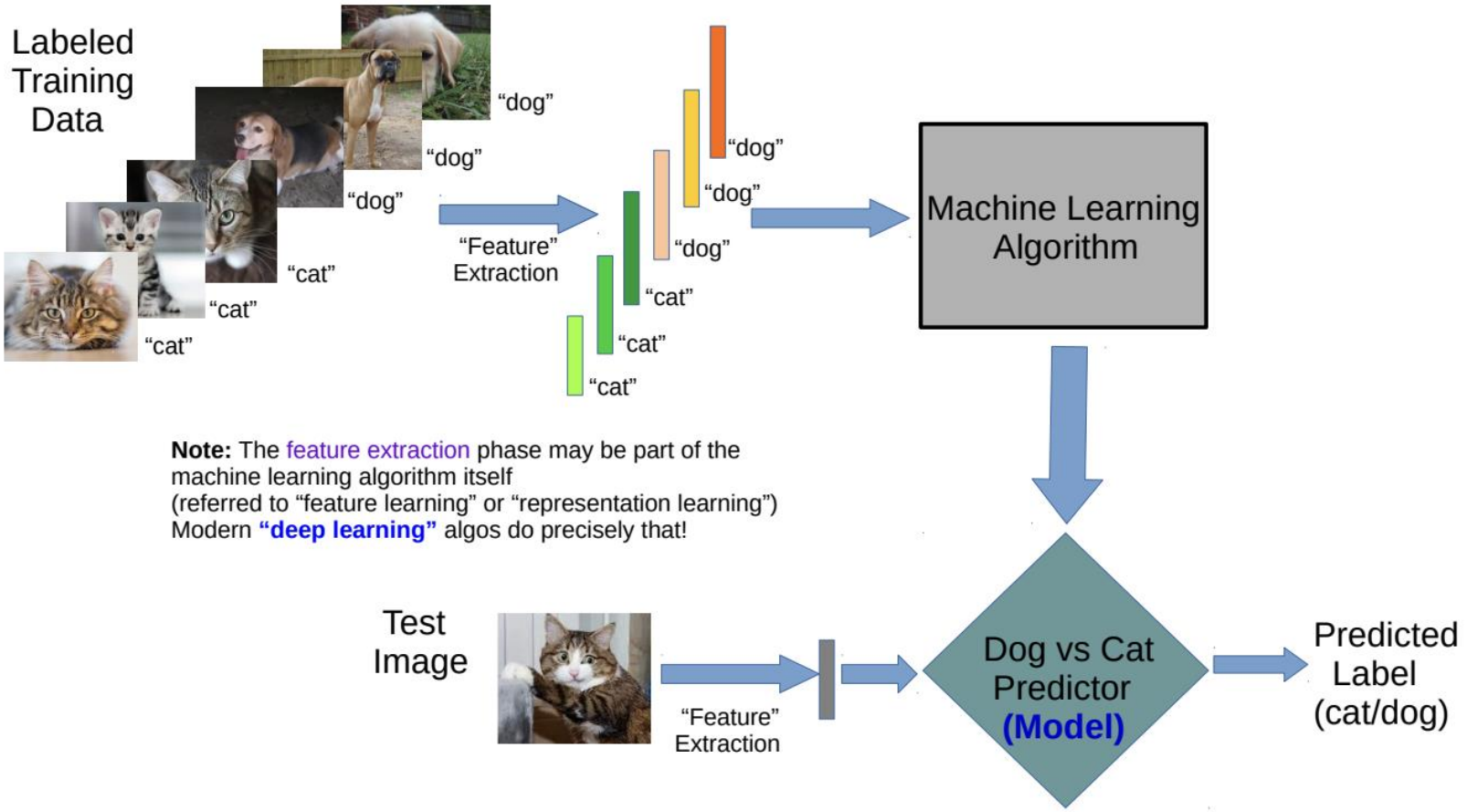
Slide from Piyush Rai

A Typical Supervised Learning Workflow (for Classification)



Slide from Piyush Rai

A Typical Supervised Learning Workflow (for Classification)



Slide from Piyush Rai

Supervised Learning

- Upon observing the data: $\mathcal{D} = \{(X_1, Y_1), (X_2, Y_2), \ldots, (X_n, Y_n)\}$, the classifier is asked to summarize what it learned about the relationship X and Y
- The learner's summary takes the form of a function $\hat{f}_D \mathcal{X} \longmapsto \mathcal{Y}$

Types of features/Types of outputs

Features (in vector x_n) as well as outputs y_n can be real-valued, binary, categorical, ordinal, etc.

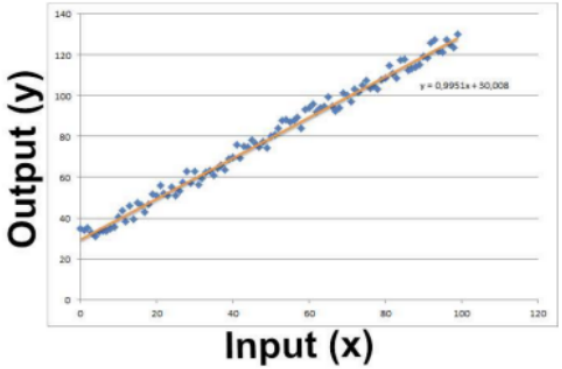
- **Real-valued:** Pixel intensity, house area, rainfall amount, temperature, etc.
- **Binary:** adult/non-adult, or any present/absent type values
- **Categorical/Discrete:** Bloodgroup, or any "which one from this finite set" type values
- **Ordinal:** Grade (A/B/C etc.) in a course, (Small/Medium/Large) in clothing sizing or any other type where relative values matters
- Often, the features can be of mixed types (some real, some categorical, some ordinal, etc.)
- Appropriate handling of different types of features may be very important
- Different types of outputs may require different type of learning models

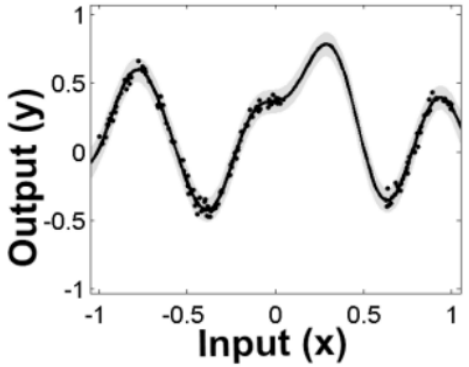
Geometric View of Some Basic ML Problems

Regression

Supervised Learning: Learn a line/curve (the "model") using training data consisting of Input-output pairs (each output is a real-valued number)

Use it to predict the outputs for new "test" inputs

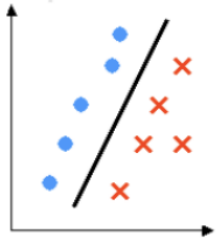




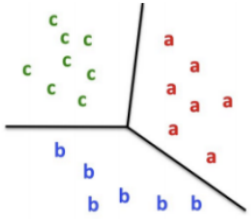
Classification

Supervised Learning: Learn a linear/nonlinear separator (the "model") using training data consisting of input-output pairs (each output is discrete-valued "label" of the corresponding input)

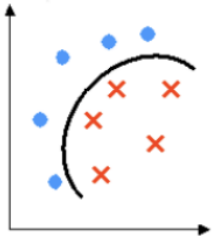
Use it to predict the labels for new "test" inputs



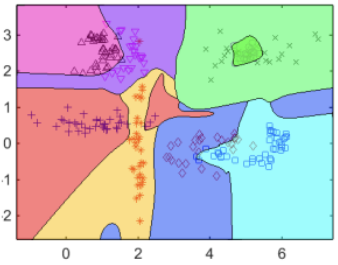
Two-Class (binary) Linear Classification



Multi-Class Linear Classification



Two-Class (binary) Nonlinear Classification



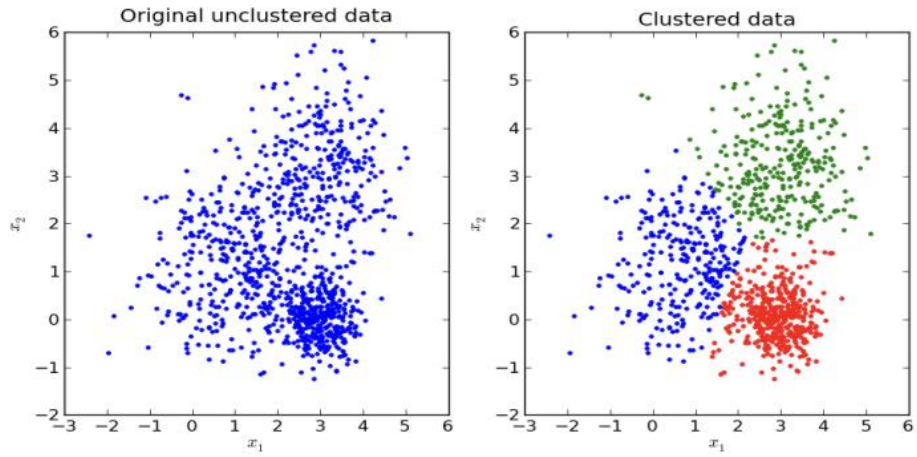
Multi-Class Nonlinear Classification

Slide from Piyush Rai

Geometric View of Some Basic ML Problems

Clustering

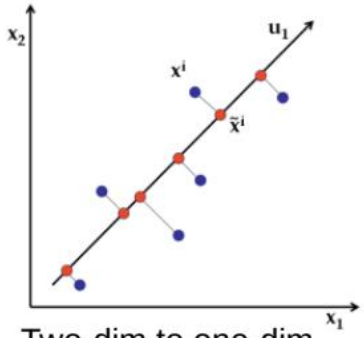
Unsupervised Learning: Learn the grouping structure for a given set of unlabeled inputs



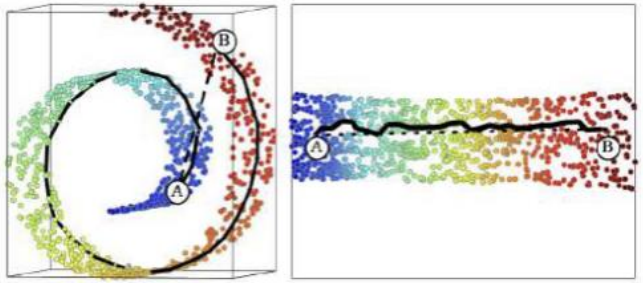
Dimensionality Reduction

Unsupervised Learning: Learn a Low-dimensional representation for a given set of high-dimensional inputs

Note: DR also comes in supervised flavors (supervised DR)

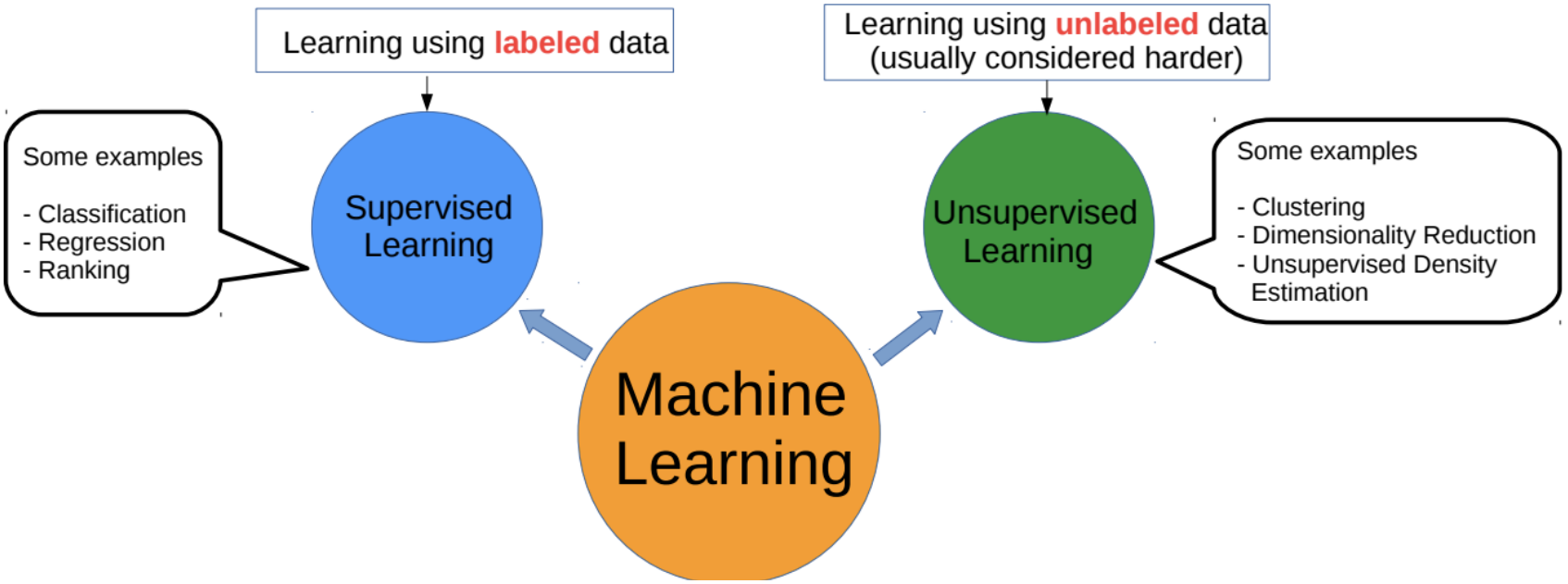


Two-dim to one-dim linear projection

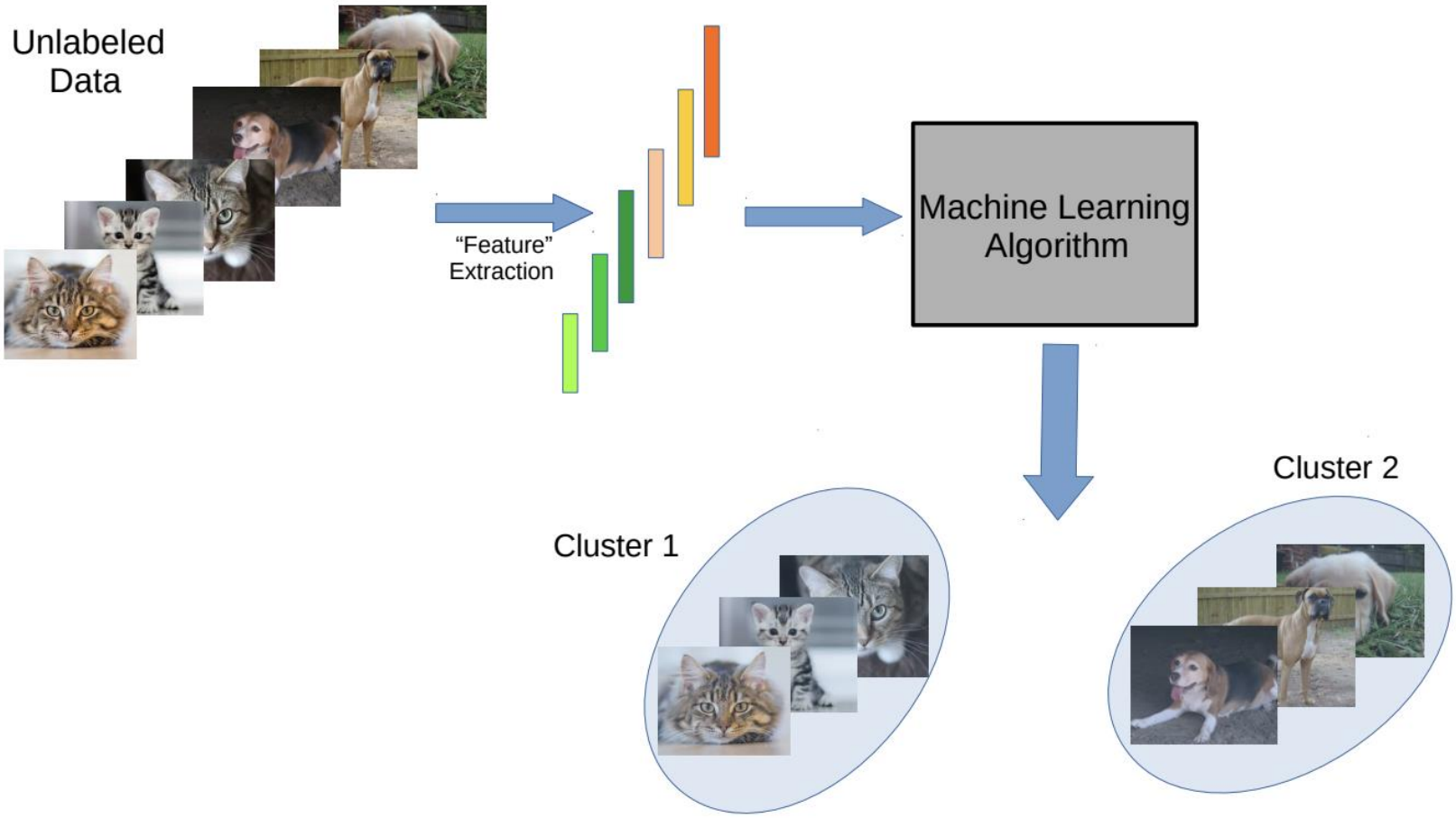


Three-dim to two-dim nonlinear projection (a.k.a. manifold learning)

A loose taxonomy of ML

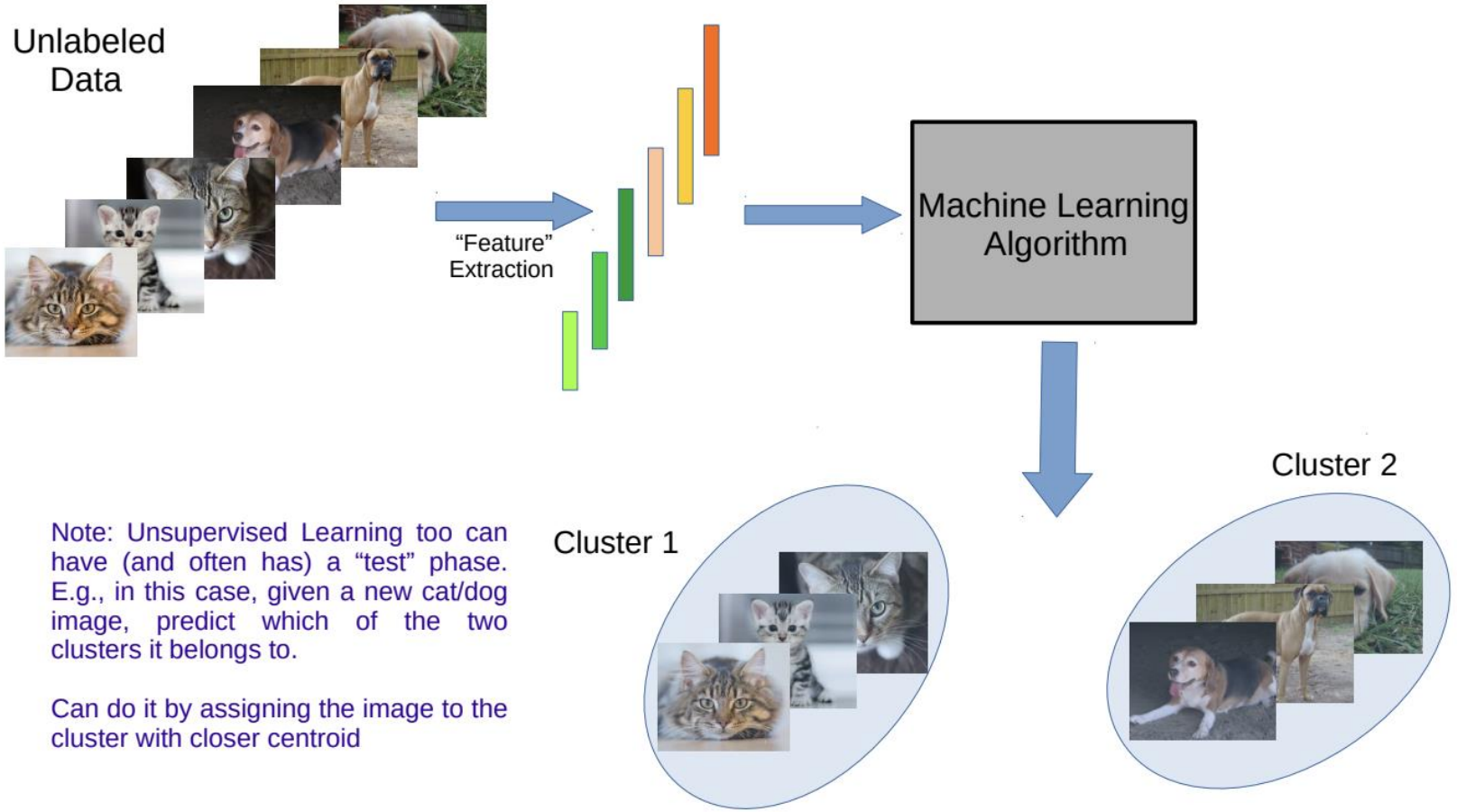


A Typical Unsupervised Learning Workflow (for Clustering)



Slide from Piyush Rai

A Typical Unsupervised Learning Workflow (for Clustering)

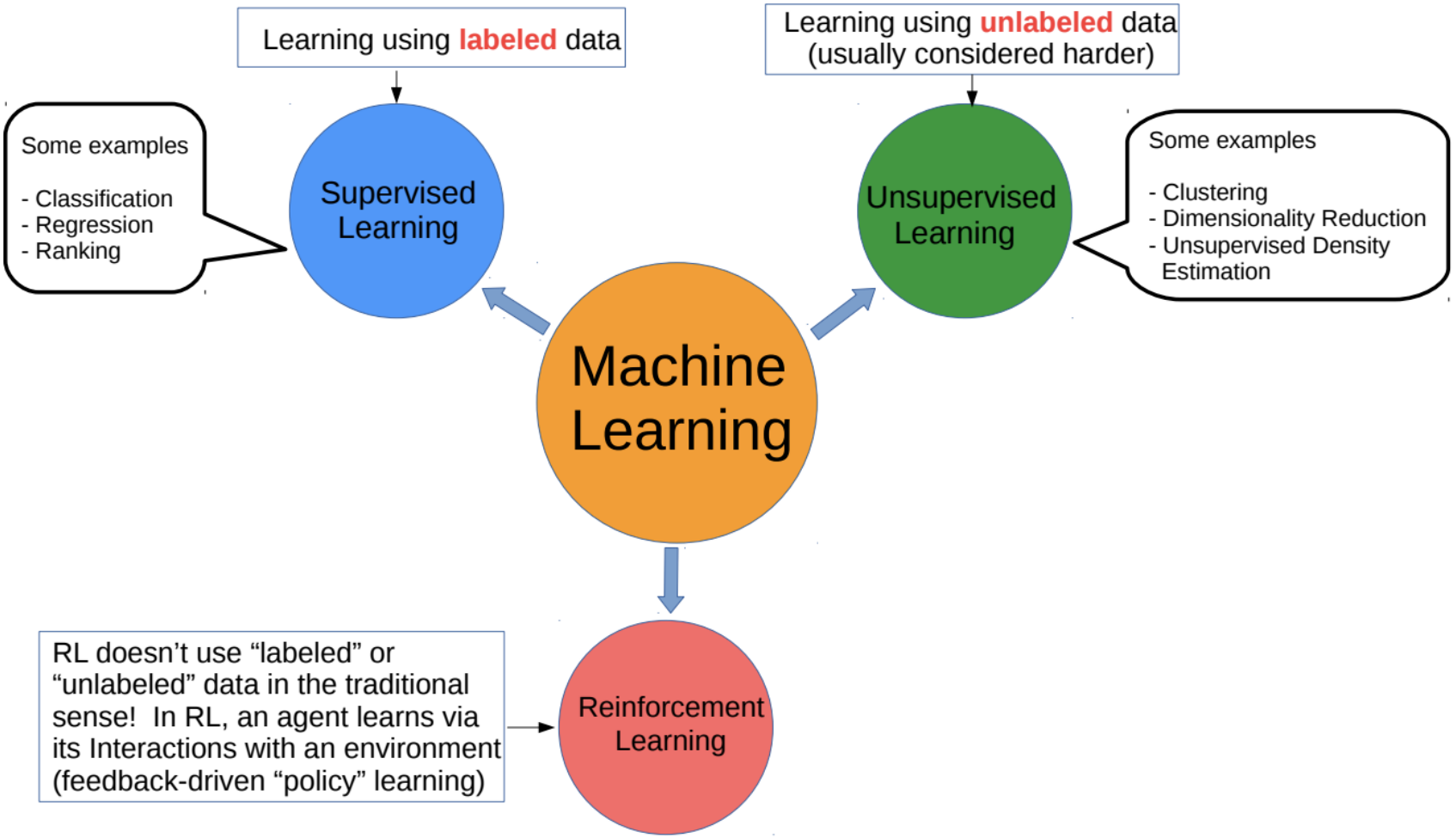


Note: Unsupervised Learning too can have (and often has) a "test" phase. E.g., in this case, given a new cat/dog image, predict which of the two clusters it belongs to.

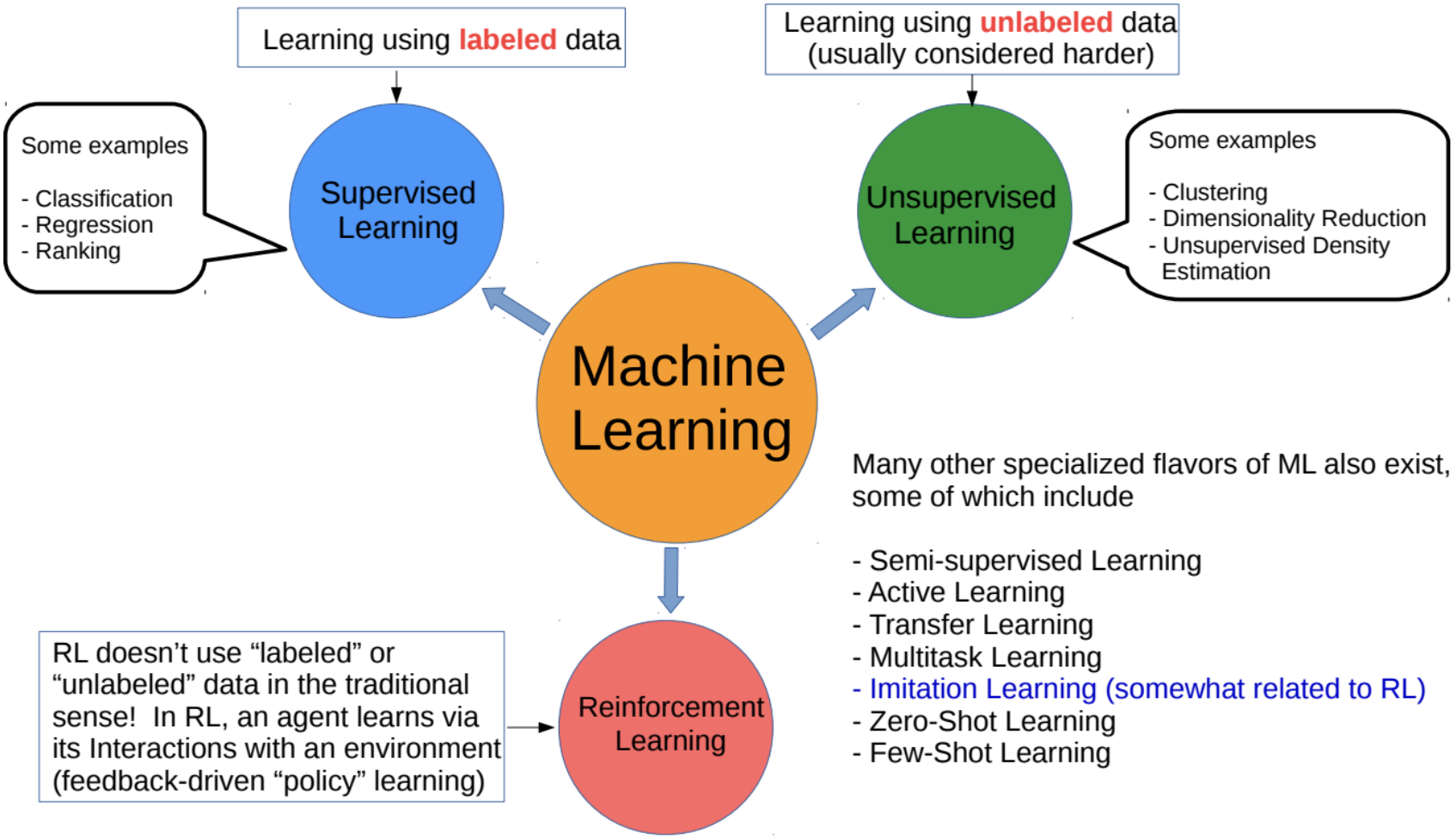
Can do it by assigning the image to the cluster with closer centroid

Slide from Piyush Rai

A loose taxonomy of ML

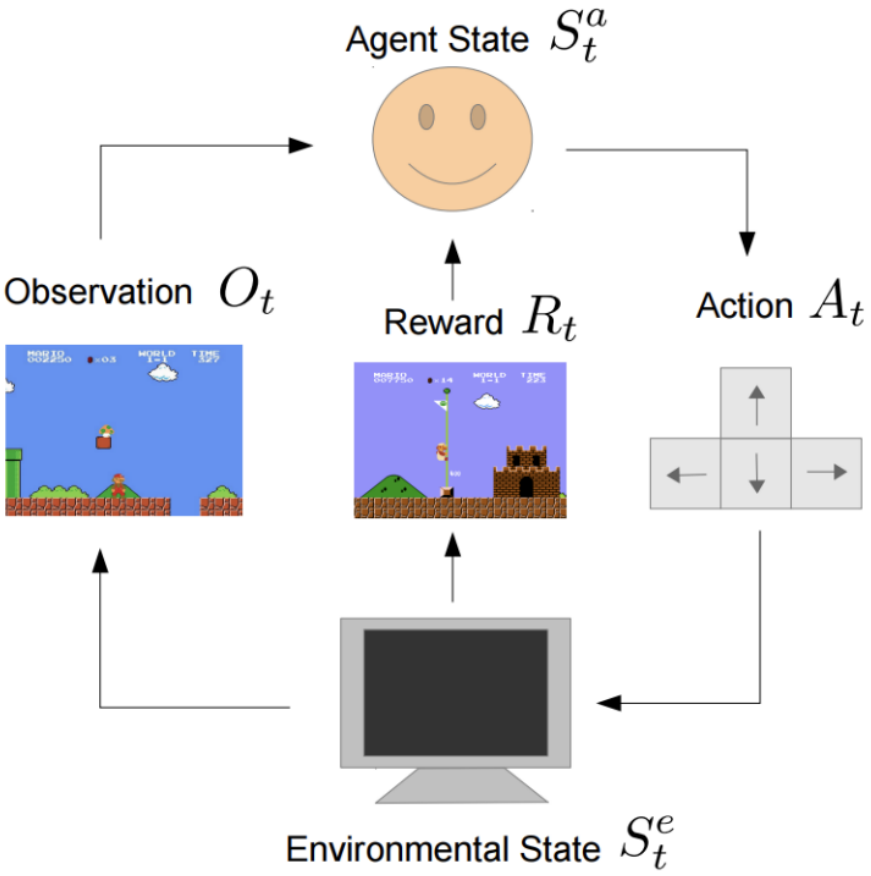


A loose taxonomy of ML



Slide from Piyush Rai

A Typical Reinforcement Learning Workflow



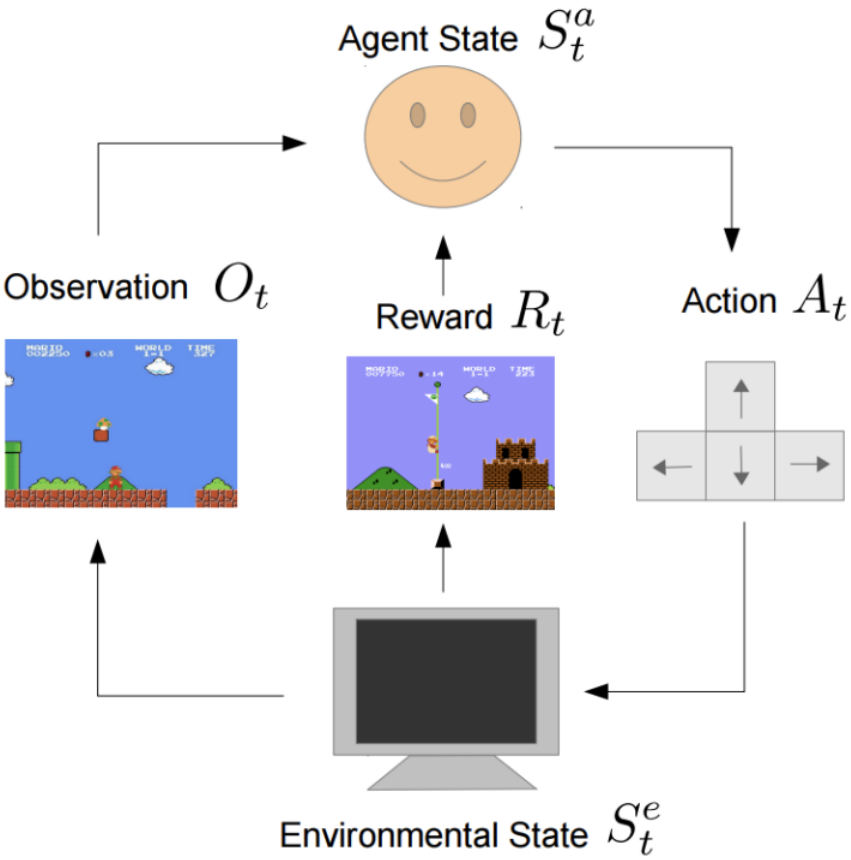
Agent's goal is to learn a policy for some task

Agent does the following repeatedly

- Senses/observes the environment
- Takes an action based on its current policy
- Receives a reward for that action
- Updates its policy

Slide from Piyush Rai

A Typical Reinforcement Learning Workflow



Agent's goal is to learn a policy for some task

Agent does the following repeatedly

- Senses/observes the environment
- Takes an action based on its current policy
- Receives a reward for that action
- Updates its policy

There IS supervision, not explicit (as in Supervised Learning) but rather implicit (feedback based)

Slide from Piyush Rai

Learning settings by training signal

Setting	Training signal	Example
Supervised	Inputs with target labels	Digit recognition
Semi-supervised	Few labeled, many unlabeled examples	Image classification with limited labels
Weakly supervised	Noisy labels	Text labeled by heuristic rules
Self-supervised	Target derived from the data	Next-token prediction
Unsupervised	No task-specific target labels	Clustering samples
Reinforcement	Rewards from interaction	Robot control

Additional learning settings

Setting	Main idea
Active learning	The model selects examples for a person to label.
Online learning	The model updates as new examples arrive.
Continual learning	The model learns new tasks while retaining prior knowledge.
Transfer learning	Learning on one task helps with another.
Multitask learning	One model learns several related tasks.
Few-shot learning	The model performs a task from very few examples.
Federated learning	Training uses data stored at separate locations.
Imitation learning	A policy learns from expert demonstrations.

These settings can overlap with those on the previous slide.

Acknowledgements

- Alex Smola, Alexander Ihler, Aarthi Singh, Tom Mitchell, Jeff Howbert, Victor Lavrenko